

Modern Computer Architecture (CSA222)

One-Shot Revision Notes

Every concept, formula and derivation from Units 1-15, step by step. Read top to bottom the day before the exam.

Yellow box = formula / rule to memorise

Derivation = where a formula comes from, every step written out

Worked = a full calculation, nothing skipped

Red = exam trap (where marks are usually lost)

- | | |
|---|--|
| 1. Boolean Algebra & Gates | 9. Von Neumann vs Harvard |
| 2. K-maps & Universal Gates | 10. CISC vs RISC |
| 3. MUX, Decoders, Adders, ALU | 11. MIPS Registers & Instruction Formats |
| 4. Number Systems & Two's Complement | 12. C to MIPS |
| 5. Latches, Flip-Flops & Timing | 13. Single-Cycle Datapath |
| 6. Registers, Counters, Shift Registers | 14. Pipelining (bonus) |
| 7. Register Files & FSMs | 15. Hazards (bonus) |
| 8. SRAM, DRAM & Memory Hierarchy | 16. All formulas on one page |

1. Boolean Algebra & Gates

Big picture

Abstraction stack (top to bottom): AI/ML models → applications & languages → OS → **ISA** → datapath & control → logic gates → transistors. The ISA is the contract between software and hardware.

Architecture = the “what” (programmer-visible: instructions, registers). **Organisation** = the “how” (hidden implementation: pipelines, caches, how ADD is built).

1 = TRUE = HIGH (about 3.3 V / 5 V); 0 = FALSE = LOW (0 V).

The seven gates

A B	AND AB	OR A+B	NAND (AB)'	NOR (A+B)'	XOR A⊕B	XNOR (A⊕B)'
0 0	0	0	1	1	0	1
0 1	0	1	1	0	1	0
1 0	0	1	1	0	1	0
1 1	1	1	0	0	0	1

NOT: A' (flips). **XOR** = **A'B + AB'** (1 when inputs DIFFER). **XNOR** = **A'B' + AB** (1 when inputs are EQUAL).

Memory hooks: AND = “all 1”; OR = “any 1”; NAND/NOR = AND/OR then flip; XOR = “odd number of 1s”.

Boolean laws

Law	OR form	AND form
Identity	$A + 0 = A$	$A \cdot 1 = A$
Null	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$A \cdot A = A$
Complement	$A + A' = 1$	$A \cdot A' = 0$
Involution	$(A')' = A$	
Distributive	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption	$A + AB = A$	$A(A + B) = A$
Redundancy	$A + A'B = A + B$	$A(A' + B) = AB$
De Morgan	$(A + B)' = A'B'$	$(AB)' = A' + B'$
Consensus	$XY + X'Z + YZ = XY + X'Z$	

De Morgan in words: “**break the bar, change the sign**”.

“+” means OR, not addition: $A + 1 = 1$ (not 2). The distributive law $A + BC = (A + B)(A + C)$ has no equivalent in ordinary algebra — students forget it exists.

Derivations of the laws you must be able to prove

Absorption: $A + AB = A$

1. Take A out as a common factor: $A + AB = A \cdot 1 + A \cdot B = A(1 + B)$
2. Null law: $1 + B = 1$
3. So $A(1 + B) = A \cdot 1 = \mathbf{A}$

Absorption (AND form): $A(A + B) = A$

1. Multiply out: $A \cdot A + A \cdot B$
2. Idempotent: $A \cdot A = A$, so we get $A + AB$
3. By the previous result, $A + AB = \mathbf{A}$

Redundancy: $A + A'B = A + B$

1. Use the distributive law $X + YZ = (X + Y)(X + Z)$ with $X = A$, $Y = A'$, $Z = B$
2. $A + A'B = (A + A')(A + B)$
3. Complement: $A + A' = 1$
4. So it is $1 \cdot (A + B) = \mathbf{A + B}$

Combining: $AB + AB' = A$

1. Take A out: $A(B + B')$
2. $B + B' = 1$
3. $A \cdot 1 = \mathbf{A}$

Consensus: $XY + X'Z + YZ = XY + X'Z$

1. Multiply the extra term YZ by $1 = (X + X')$: $YZ = YZ(X + X') = XYZ + X'YZ$
2. Whole expression = $XY + X'Z + XYZ + X'YZ$
3. Group: $(XY + XYZ) + (X'Z + X'YZ)$
4. Absorption on each group: $XY(1 + Z) = XY$ and $X'Z(1 + Y) = X'Z$
5. Result = $\mathbf{XY + X'Z}$ (the YZ term was redundant)

Worked simplifications

Simplify $A'B'C + A'BC + AB'C + ABC + A'B$

1. Look at the first four terms: they all contain C. Take C out: $C(A'B' + A'B + AB' + AB)$
2. $A'B' + A'B + AB' + AB$ is every combination of A and B, so it equals 1. (Check: $A'(B' + B) + A(B' + B) = A' + A = 1$)
3. So the first four terms = $C \cdot 1 = C$
4. Expression = $\mathbf{C + A'B}$

Complement of $(A + B)(A' + C)$

1. First simplify: $(A + B)(A' + C) = AA' + AC + A'B + BC = 0 + AC + A'B + BC$
2. BC is the consensus of AC and $A'B$, so it disappears $\rightarrow AC + A'B$
3. Complement: $(AC + A'B)' = (AC)' \cdot (A'B)'$ (De Morgan)
4. $= (A' + C')(A + B')$ (De Morgan again on each bracket)
5. Multiply out: $A'A + A'B' + AC' + B'C' = 0 + A'B' + AC' + B'C'$
6. $B'C'$ is the consensus of $A'B'$ and AC' $\rightarrow$ drops. Answer **$A'B' + AC'$**

Nested De Morgan: $((A + B)'C)' + (AB)'$

1. $((A + B)'C)' = ((A + B)')' + C' = (A + B) + C'$
2. $(AB)' = A' + B'$
3. Sum = $A + B + C' + A' + B'$
4. $A + A' = 1$, and $1 + \text{anything} = 1$. Answer **1**

Lab essentials (Verilog)

```
module and_gate(input a, input b, output y); assign y = a & b; endmodule
```

Operators: `&` AND, `|` OR, `~` NOT, `^` XOR, `~^` XNOR. `wire` = driven by `assign`; `reg` = assigned inside `always / initial`.

Testbench: no ports; `reg` inputs, `wire` outputs; `#10` = wait 10 time units; `$monitor` prints on change; `$dumpfile/$dumpvars` $\rightarrow$ waveform; `$finish` ends.

Simulation (iverilog $\rightarrow$ GTKWave) checks behaviour; synthesis turns RTL into a gate netlist.

Flow: spec $\rightarrow$ RTL $\rightarrow$ simulate $\rightarrow$ synthesize $\rightarrow$ netlist $\rightarrow$ place & route $\rightarrow$ verify (DRC/LVS/STA) $\rightarrow$ fabricate.

2. K-maps & Universal Gates

Why minimise: fewer gates → less area, less power, faster, cheaper ("16 gates → 2 gates").

Canonical forms

Minterm (for SOP): a row where $F = 1$. Variable 1 → write X , variable 0 → write X' .

Example row 101 → $m_5 = AB'C$.

Maxterm (for POS): a row where $F = 0$. Variable 0 → write X , variable 1 → write X' .

Example row 011 → $M_3 = (A + B' + C')$.

$F = \sum m(\text{rows where } F = 1) = \prod M(\text{all the other rows})$. The two index lists together are 0 to $2^n - 1$.

The POS rule is the **OPPOSITE** of the SOP rule (0 → plain variable). This flip is the number-one trap.

$F = \sum m(1, 3, 5, 7)$ for A, B, C

1. Rows 001, 011, 101, 111 → $SOP = A'B'C + A'BC + AB'C + ABC$
2. POS uses the other rows 0, 2, 4, 6 → $(A + B + C)(A + B' + C)(A' + B + C)(A' + B' + C)$
3. Simplify the SOP: $A'C(B' + B) + AC(B' + B) = A'C + AC$
4. $= C(A' + A) = \mathbf{C}$. (Every listed row has $C = 1$ and the others have $C = 0$.)

Karnaugh maps

Rows/columns are in **Gray code: 00, 01, 11, 10** (neighbours differ in ONE bit).

Group only 1s (and don't-cares) in rectangles of size **1, 2, 4, 8, 16** (powers of 2). The map **wraps around**: left edge touches right edge, top touches bottom, the four corners form one group.

A group of 2^k cells removes k variables. Keep only the variables that do not change inside the group.

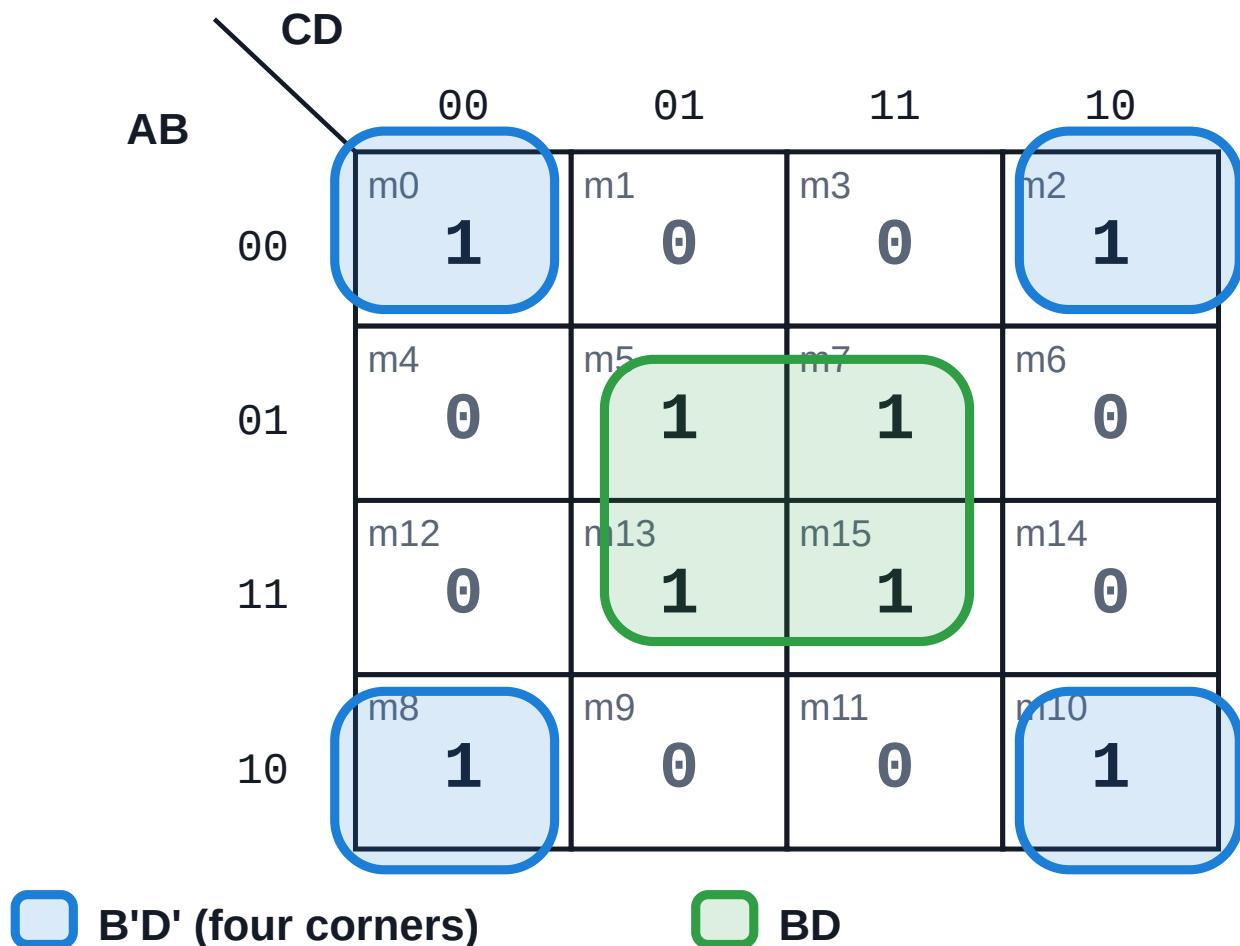
Make groups as **LARGE** as possible and as **FEW** as possible; overlap is allowed; diagonal groups are not.

Why grouping works

1. Two cells side by side differ in exactly one variable, say B : e.g. $A'BC'$ and $A'BC$.
2. Add them: $A'C'(B + B') = A'C'$. The changing variable B disappears.
3. A group of 4 = two such pairs differing in another variable → that one disappears too. So each doubling removes one variable.

$F(A,B,C,D) = \Sigma m(0, 2, 5, 7, 8, 10, 13, 15)$

1. Corners m_0 (0000), m_2 (0010), m_8 (1000), m_{10} (1010): in all four, $B = 0$ and $D = 0$; A and C change $\rightarrow$ group = **$B'D'$**
2. Centre m_5 (0101), m_7 (0111), m_{13} (1101), m_{15} (1111): $B = 1$ and $D = 1$ everywhere; A and C change $\rightarrow$ **BD**
3. $F = B'D' + BD = (\mathbf{B} \odot \mathbf{D})$, the XNOR of B and D.



D2.3d · Wrap-around: $\Sigma m(0,2,5,7,8,10,13,15) = B'D' + BD$

$F(A,B,C,D) = \Sigma m(0, 1, 2, 5, 8, 9, 10)$

1. Corners 0, 2, 8, 10 $\rightarrow B'D'$
2. m_0, m_1, m_8, m_9 ($B = 0, C = 0$; A and D change) $\rightarrow B'C'$
3. m_5 (0101) is still uncovered. Pair it with m_1 (0001): $A = 0, C = 0, D = 1$ stay fixed $\rightarrow A'C'D$
4. $F = B'D' + B'C' + A'C'D$

Don't-cares (X): input combinations that never happen (e.g. BCD 10–15). Treat an X as 1 if it makes a group bigger, otherwise as 0; never group Xs alone.

Prime implicant (PI) = a group that cannot be made larger. **Essential PI (EPI)** = a PI that is the only one covering some 1. The minimal SOP = all EPIs + the fewest extra PIs to cover the remaining 1s. A **cyclic** map (e.g. $\Sigma m(0,1,2,5,6,7)$) has no EPI: answer $A'B' + BC' + AC$.

De Morgan & bubble pushing

NAND = OR with inverted inputs: $(AB)' = A' + B'$. NOR = AND with inverted inputs: $(A + B)' = A'B'$.

Two bubbles on the same wire cancel (double inversion).

Universal gates — gate counts

	NOT	AND	OR	XOR
NAND only	1	2	3	4
NOR only	1	3	2	5

SOP (AND-OR) → **NAND-NAND**. POS (OR-AND) → **NOR-NOR**. $(A + B)(C + D)$ with only 2-input NANDs needs **6**.

CMOS: NAND = 4 transistors, AND = 6 (AND is NAND + inverter) → fabs prefer NAND.

Building each gate from NAND

1. NOT: $\text{NAND}(A, A) = (A \cdot A)' = A'$
2. AND: NAND then NOT: $((AB)')' = AB \rightarrow 2 \text{ gates}$
3. OR: $\text{NAND}(A', B') = (A'B')' = A + B$ (De Morgan). A' and B' each cost one NAND → 3 gates

XOR from 4 NANDs

1. Gate 1: $X = (AB)' = A' + B'$
2. Gate 2: $P = (A \cdot X)'$. Inside: $A \cdot X = A(A' + B') = AA' + AB' = AB'$. So $P = (AB')'$
3. Gate 3: $Q = (B \cdot X)'$. Inside: $B(A' + B') = A'B$. So $Q = (A'B)'$
4. Gate 4: $(P \cdot Q)' = P' + Q' = AB' + A'B = \mathbf{A \oplus B}$

Why SOP becomes NAND-NAND: $F = AB + CD$

1. Double complement changes nothing: $F = ((AB + CD)')'$
2. De Morgan on the inner bar: $(AB + CD)' = (AB)' \cdot (CD)'$
3. So $F = ((AB)' \cdot (CD)')'$ → a NAND of two NANDs. Three 2-input NANDs.
4. Same idea for POS: $F = (A + B)(C + D) = ((A + B)' + (C + D)')'$ → NOR of two NORs.

3. MUX, Decoders, Adders, ALU

Multiplexer (data selector)

2:1 MUX: $Y = S' \cdot I_0 + S \cdot I_1$

4:1 MUX: $Y = S_1' S_0' \cdot I_0 + S_1' S_0 \cdot I_1 + S_1 S_0' \cdot I_2 + S_1 S_0 \cdot I_3$

n select lines $\rightarrow 2^n$ inputs (8:1 needs 3 selects). A $2^n:1$ MUX built from 2:1 MUXes needs $2^n - 1$ of them (4:1 $\rightarrow 3$; 8:1 $\rightarrow 7$).

A $2^n:1$ MUX can implement ANY function of n variables; a $2^n:1$ MUX can also do n + 1 variables using the "residue" trick (inputs 0, 1, X or X').

Implement $F(A,B,C) = \Sigma m(1, 3, 5, 6)$ with a 4:1 MUX ($S_1 = A, S_0 = B$)

1. For each value of AB, see what F does as C changes.
2. AB = 00: rows 000 (m0, F = 0) and 001 (m1, F = 1) $\rightarrow F = C \rightarrow I_0 = C$
3. AB = 01: rows 010 (m2, 0) and 011 (m3, 1) $\rightarrow F = C \rightarrow I_1 = C$
4. AB = 10: rows 100 (m4, 0) and 101 (m5, 1) $\rightarrow F = C \rightarrow I_2 = C$
5. AB = 11: rows 110 (m6, 1) and 111 (m7, 0) $\rightarrow F = C' \rightarrow I_3 = C'$

XOR on a 4:1 MUX with $S_1 = A, S_0 = B$: $I_0 = 0, I_1 = 1, I_2 = 1, I_3 = 0$.

Demultiplexer and decoder

1:2 DEMUX: $Y_0 = S' \cdot D, Y_1 = S \cdot D$.

n-to- 2^n decoder: output **D_k = minterm k**; exactly one output is 1 (one-hot); built from 2^n AND gates + n inverters.

A decoder is a DEMUX whose data input is tied to 1 (the enable).

Any function = OR of the decoder outputs for its minterms.

	MUX	DEMUX	Decoder
Inputs	2^n data + n select	1 data + n select	n address
Outputs	1	2^n	2^n (one-hot)
Job	many $\rightarrow$ one	one $\rightarrow$ many	number $\rightarrow$ one line

Full adder from a 3-to-8 decoder: Sum = $D_1 + D_2 + D_4 + D_7$; Cout = $D_3 + D_5 + D_6 + D_7$.

Half adder

From the truth table

1. A B $\rightarrow$ Sum Carry: 00 $\rightarrow$ 0 0; 01 $\rightarrow$ 1 0; 10 $\rightarrow$ 1 0; 11 $\rightarrow$ 0 1 (1 + 1 = 10 in binary)
2. Sum is 1 when the inputs differ $\rightarrow$ Sum = $A'B + AB' = A \oplus B$
3. Carry is 1 only for 11 $\rightarrow$ **Carry = $A \cdot B$**
4. Limitation: no carry-in, so it cannot be chained.

Full adder

Sum = $A \oplus B \oplus \text{Cin}$ (odd parity) **Cout = $AB + \text{Cin}(A \oplus B)$** = $AB + B\text{Cin} + A\text{Cin}$ (majority)

Sum derivation

1. Sum = 1 on rows 001, 010, 100, 111 $\rightarrow \Sigma m(1, 2, 4, 7) = A'B'C + A'BC' + AB'C' + ABC$ ($C = \text{Cin}$)
2. Group by A: $A'(B'C + BC') + A(B'C' + BC)$
3. $B'C + BC' = B \oplus C$, and $B'C' + BC = (B \oplus C)'$
4. So Sum = $A'(B \oplus C) + A(B \oplus C)'$ — this is the XOR pattern $X'Y + XY'$ with $X = A$, $Y = B \oplus C$
5. Sum = **$A \oplus B \oplus C$**

Cout derivation

1. Cout = 1 on rows 011, 101, 110, 111 $\rightarrow A'BC + AB'C + ABC' + ABC$
2. Group the last two: $ABC' + ABC = AB(C' + C) = AB$
3. Group the first two: $A'BC + AB'C = C(A'B + AB') = C(A \oplus B)$
4. Cout = **$AB + C(A \oplus B)$**

Full adder from two half adders + OR: $\text{HA1}(A, B) \rightarrow S1 = A \oplus B, C1 = AB$. $\text{HA2}(S1, \text{Cin}) \rightarrow \text{Sum} = S1 \oplus \text{Cin}, C2 = S1 \cdot \text{Cin}$. $\text{Cout} = C1 + C2$. The OR is safe because $C1$ and $C2$ can never both be 1 (if $A = B = 1$ then $S1 = 0$ so $C2 = 0$).

Ripple-carry adder and carry-lookahead

n -bit RCA = n full adders, Cout of stage $i \rightarrow \text{Cin}$ of stage $i + 1$, $\text{Cin}_0 = 0$ (or 1 for subtraction).

Delay grows **linearly**: about 2 gate delays per bit $\rightarrow$ 4-bit ≈ 8 , 64-bit ≈ 128 gate delays.

Carry-lookahead: Generate $G_i = A_i \cdot B_i$, Propagate $P_i = A_i \oplus B_i$, **$C(i+1) = G_i + P_i \cdot C_i$** .

Expanding the lookahead carries (no waiting for ripple)

1. $C1 = G0 + P0 \cdot C0$
2. $C2 = G1 + P1 \cdot C1 = G1 + P1(G0 + P0C0) = G1 + P1G0 + P1P0C0$
3. $C3 = G2 + P2 \cdot C2 = G2 + P2G1 + P2P1G0 + P2P1P0C0$
4. Every carry is now a 2-level AND-OR of the inputs $\rightarrow$ roughly constant delay per block, at the cost of more area.

ALU (preview, researched)

1-bit ALU slice = AND gate + OR gate + full adder + MUX selecting the operation. Subtract with $\text{Binvert} = 1$ and carry-in = 1 ($A + B' + 1$).

ALU control: 0000 AND, 0001 OR, 0010 add, 0110 subtract, 0111 slt. Zero flag = NOR of all result bits (1 when the result is 0).

4. Number Systems & Two's Complement

Conversions

Value = $\Sigma (\text{digit} \times \text{base}^{\text{position}})$, positions counted from 0 on the right.

Powers of 2: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024. 1 hex digit = 4 bits; 1 octal digit = 3 bits.

Hex digits: A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

Binary → decimal: 101101

1. Write the weights under the bits: $1 \times 32, 0 \times 16, 1 \times 8, 1 \times 4, 0 \times 2, 1 \times 1$
2. Add the weights where the bit is 1: $32 + 8 + 4 + 1 = 45$

Decimal → binary: 42

1. $42 \div 2 = 21$ remainder 0
2. $21 \div 2 = 10$ remainder 1
3. $10 \div 2 = 5$ remainder 0
4. $5 \div 2 = 2$ remainder 1
5. $2 \div 2 = 1$ remainder 0
6. $1 \div 2 = 0$ remainder 1
7. Read the remainders from the BOTTOM up: **101010**. Check: $32 + 8 + 2 = 42$ ✓

Binary ↔ hex: 10110101

1. Split into groups of 4 from the right: 1011 | 0101
2. $1011 = 8 + 2 + 1 = 11 = \text{B}$; $0101 = 4 + 1 = 5 \rightarrow \text{0xB5}$
3. To decimal: $\text{B} \times 16 + 5 = 11 \times 16 + 5 = 176 + 5 = 181$

Also: $0x2A = 0010\ 1010 = 2 \times 16 + 10 = 42$. $0xFF = 255$.

Signed representations

n bits	Range	Zeros
Unsigned	0 to $2^n - 1$	one
Sign-magnitude	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$	two (+0, -0)
One's complement	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$	two
Two's complement	-2^{n-1} to $2^{n-1} - 1$	one

Two's complement ranges: 4-bit -8 to 7; 6-bit -32 to 31; 8-bit -128 to 127; 16-bit -32768 to 32767; 32-bit about ± 2.1 billion.

MSB weight is -2^{n-1} (-128 in 8 bits); all other bits are positive.

Two's complement

Negate: invert every bit, then add 1 ($-x = x' + 1$).

Shortcut: from the right, copy bits up to and including the first 1, then invert everything to its left.

Why “invert and add 1” gives $-x$

1. In n bits, $x + x' = 111\dots 1$ (every column is $0 + 1$ or $1 + 0$) = $2^n - 1$
2. So $x' + 1 = 2^n - x$
3. An n -bit register throws away the 2^n carry (works modulo 2^n), so $2^n - x$ behaves exactly like $-x$

-18 in 8 bits

1. $+18 = 16 + 2 = 00010010$
2. Invert: 11101101
3. Add 1: $11101110 \rightarrow \mathbf{-18 = 11101110}$
4. Check with the MSB weight: $-128 + 64 + 32 + 8 + 4 + 2 = -128 + 110 = -18 \checkmark$

-45 in 8 bits

1. $45 = 32 + 8 + 4 + 1 = 00101101$
2. Invert: 11010010
3. Add 1: **11010011**

0xB7 as signed and unsigned

1. $0xB7 = 1011\ 0111$
2. Unsigned: $128 + 32 + 16 + 4 + 2 + 1 = 183$
3. Signed: $-128 + 32 + 16 + 4 + 2 + 1 = -128 + 55 = \mathbf{-73}$

Same bits, different meaning: $11111111 = 255$ unsigned = -1 signed. $10000000 = -128$ (the most negative 8-bit value).

Sign extension

To widen a two's-complement number, **copy the MSB (sign bit) into every new bit**. $+5 = 0101 \rightarrow 0000\ 0101$; $-5 = 1011 \rightarrow 1111\ 1011$. Zero-padding -5 would give $+11$ (wrong). MIPS: 16-bit $0xFFFFB \rightarrow 0xFFFFFFFFB = -5$. Unsigned values (`lbu`, `andi/ori/xori`) are zero-extended.

Why copying the sign bit keeps the value (negative case)

1. Old MSB of an n -bit number has weight -2^{n-1}
2. After adding one more 1 on top: the new top bit has weight -2^n and the old top bit becomes $+2^{n-1}$
3. Their sum: $-2^n + 2^{n-1} = -2 \cdot 2^{n-1} + 2^{n-1} = -2^{n-1}$ — the same as before, so the value does not change

Addition, subtraction, overflow

$A - B = A + B' + 1$. Adder-subtractor: each B_i goes through XOR with M , and M is also the carry-in. $M = 0 \rightarrow A + B$; $M = 1 \rightarrow A + B' + 1 = A - B$.

Signed overflow $V = C(\text{into MSB}) \oplus C(\text{out of MSB}) = C_3 \oplus C_4$ for 4 bits.

Rule: overflow only when two numbers of the **same sign** give a result of the **opposite sign**. Adding numbers of opposite signs never overflows.

Unsigned overflow (carry flag CF) = C_4 , the carry out of the MSB. In subtraction ($A + B' + 1$), carry = 1 means NO borrow.

Why $V = C_3 \oplus C_4$ (look only at the MSB column: a_3 , b_3 , carry-in c_3)

1. Case $a_3 = b_3 = 0$ (both positive): sum bit = c_3 , carry out $c_4 = 0$. Overflow when $c_3 = 1$ (result looks negative) $\rightarrow c_3 = 1, c_4 = 0 \rightarrow$ they differ.
2. Case $a_3 = b_3 = 1$ (both negative): $c_4 = 1$ always; sum bit = c_3 . Overflow when $c_3 = 0$ (result looks positive) $\rightarrow c_3 = 0, c_4 = 1 \rightarrow$ they differ.
3. Case $a_3 \neq b_3$ (mixed signs): $c_4 = c_3$, never overflow $\rightarrow$ they are equal.
4. So overflow $\Leftrightarrow c_3 \neq c_4 \Leftrightarrow V = c_3 \oplus c_4$

5 - 3 in 4 bits

1. $5 = 0101$, $3 = 0011 \rightarrow$ invert 3: 1100
2. $0101 + 1100 + 1$: $0101 + 1101 = 1\ 0010$
3. Drop the carry $\rightarrow 0010 = 2 \checkmark$ (carry = 1 $\rightarrow$ no borrow)

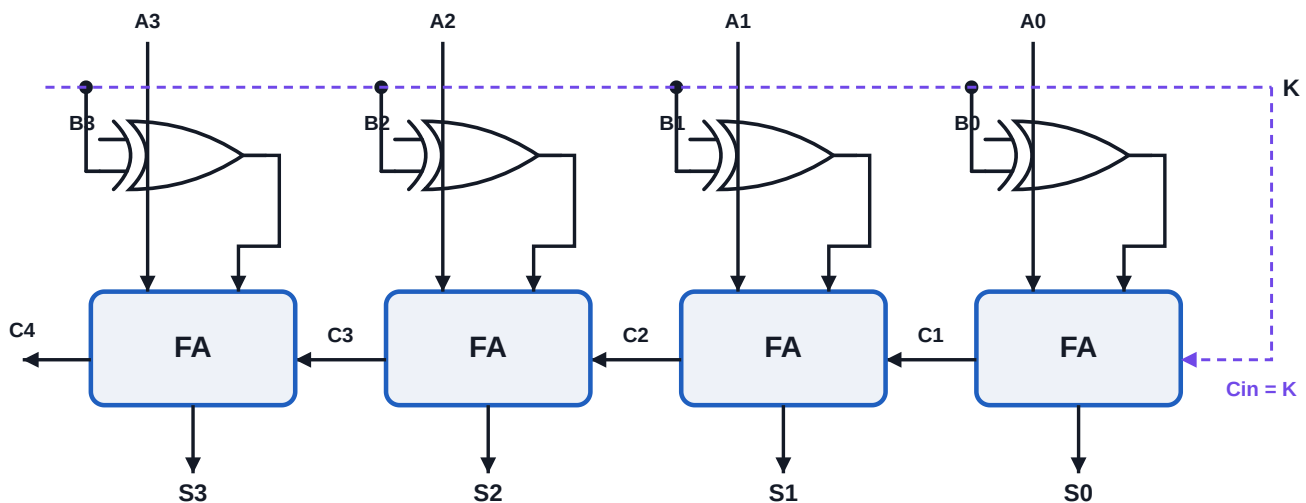
Four flag cases (4-bit, Lab 5)

Sum	Result	C3	C4 = CF	V = C3 $\oplus$ C4	Meaning
0001 + 0001	0010	0	0	0	1 + 1 = 2, fine
1111 + 0001	0000	1	1	0	unsigned 15 + 1 overflows; signed -1 + 1 = 0 fine
0111 + 0001	1000	1	0	1	signed 7 + 1 = -8 $\rightarrow$ overflow
1000 + 1000	0000	0	1	1	both overflow (-8 + -8)

Carries for 0111 + 0001 column by column: bit0 1 + 1 = 10 (carry 1); bit1 1 + 0 + 1 = 10; bit2 1 + 0 + 1 = 10 $\rightarrow$ C3 = 1; bit3 0 + 0 + 1 = 1, C4 = 0.

More overflow checks

1. 5 + 4: 0101 + 0100 = 1001 = -7 $\rightarrow$ two positives gave a negative $\rightarrow$ overflow
2. -6 + -5: 1010 + 1011 = 1 0101 $\rightarrow$ keep 0101 = +5 $\rightarrow$ two negatives gave a positive $\rightarrow$ overflow
3. 6 + (-4): 0110 + 1100 = 1 0010 $\rightarrow$ 2 $\rightarrow$ mixed signs, no overflow



K = 0: B passes, Cin = 0 $\rightarrow$ A + B. K = 1: B inverted, Cin = 1 $\rightarrow$ A + B' + 1 = A - B.

D4.5a · 4-bit adder-subtractor (Lab 5)

5. Latches, Flip-Flops & Timing

Combinational: output depends only on present inputs (adder, MUX). **Sequential:** output depends on inputs + stored state (counter, register). The difference is a **feedback loop**. Two inverters in a loop = bistable = 1-bit memory with no way to change it.

SR latch

NOR SR latch (active-HIGH): $Q = (R + Q')'$, $Q' = (S + Q)'$.

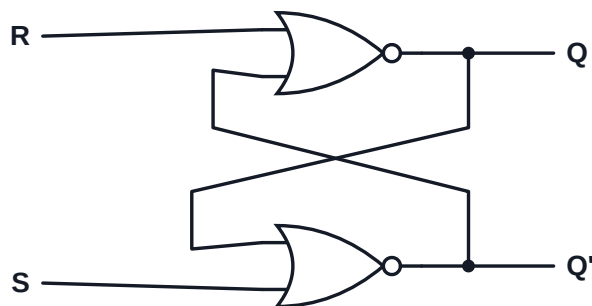
S R: 00 hold · 10 set ($Q = 1$) · 01 reset ($Q = 0$) · **11 forbidden** ($Q = Q' = 0$; releasing both together → race, unpredictable).

NAND SR latch (active-LOW S' , R'): 11 hold · 01 set · 10 reset · **00 forbidden** ($Q = Q' = 1$).

Characteristic equation: $Q^+ = S + R'Q$, with $SR = 0$.

Deriving $Q^+ = S + R'Q$

- Table of (S, R, Q) → Q^+ : hold rows 000 → 0, 001 → 1; reset rows 010 → 0, 011 → 0; set rows 100 → 1, 101 → 1; forbidden rows 110, 111 → X
- $Q^+ = 1$ on 001, 100, 101; X on 110, 111
- Group 100, 101, 110, 111 ($S = 1$) → S. Group 001 and 101 ($R = 0, Q = 1$) → $R'Q$
- $Q^+ = S + R'Q$



S	R	Q^+	action
0	0	Q	hold
1	0	1	set
0	1	0	reset
1	1	–	forbidden ($Q=Q'=0$)

NOR latch: inputs are active-HIGH. Each output feeds the other gate.

D5.2a · NOR SR latch + characteristic table

D latch (level-triggered)

$S = D \cdot EN$, $R = D' \cdot EN$ → S and R can never both be 1 → no forbidden state.

EN = 1: **transparent** (Q follows D, glitches pass). EN = 0: holds. $Q^+ = EN \cdot D + EN' \cdot Q$.

D flip-flop (edge-triggered)

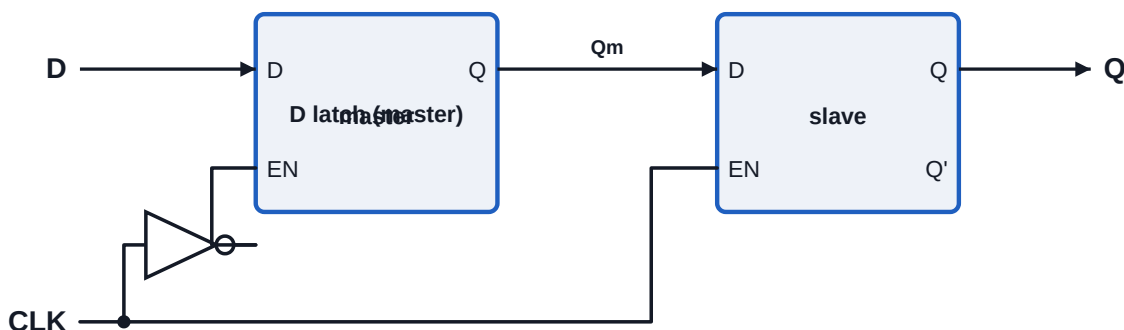
At each active (rising) edge: $Q^+ = D$; between edges Q is frozen. Triangle on the clock pin = edge-triggered.

Master-slave: two D latches on opposite clock phases. Master transparent while CLK = 0, slave while CLK = 1 → value just before the rising edge reaches Q (positive-edge). The

class slide enables the master on CLK = 1 → output changes on the FALLING edge. The two latches are never transparent together.

Verilog: `always @(posedge clk) q <= d;` (non-blocking `<=`, q declared `reg`).

	D latch	D flip-flop
Triggered by	level (EN = 1)	edge
Transparent?	yes	no
Glitches on D	pass to Q	ignored
Size	smaller	≈ 2 × a latch
Verilog	<code>always @(*) if (en) q = d;</code>	<code>always @(posedge clk) q <= d;</code>
Use	SRAM cells, time borrowing	registers, PC, pipelines



CLK = 0: master transparent (Qm follows D), slave holds. CLK 0 → 1: master closes, slave opens → Q takes D at the RISING edge.

Class slide variant: master enabled on CLK = 1 → the output changes at the FALLING edge (negative-edge FF).

D5.4c · Master-slave D flip-flop

Reading a waveform: draw a dashed line at every rising edge, read D exactly there, hold that value until the next edge. For a latch, Q copies D for the whole time EN = 1.

JK and T flip-flops

JK: 00 hold, 01 reset, 10 set, **11 toggle** → $Q^+ = JQ' + K'Q$.

T: T = 0 hold, T = 1 toggle → $Q^+ = T \oplus Q$. Build T from JK (J = K = T) or from D (D = T ⊕ Q).

Excitation table (input needed for $Q \rightarrow Q^+$):

$Q \rightarrow Q^+$	D	T	JK	SR
0 → 0	0	0	0 X	0 X
0 → 1	1	1	1 X	1 0
1 → 0	0	1	X 1	0 1
1 → 1	1	0	X 0	X 0

Shortcuts: $D = Q^+$; $T = Q \oplus Q^+$ (1 when the bit must change).

Timing

t_setup: D stable this long BEFORE the edge. **t_hold:** stable this long AFTER the edge.

t_pcq (clock-to-Q): edge → Q valid.

Setup (max-delay) check: $T_c \geq t_{pcq} + t_{pd}(\text{logic}) + t_{setup} \rightarrow f_{max} = 1 / T_c$

Hold (min-delay) check: $t_{ccq} + t_{cd}(\text{logic}) \geq t_{hold}$ — independent of the clock, so slowing the clock never fixes it; add delay to the short path.

Unit help: $1 / (1 \text{ ns}) = 1 \text{ GHz}$; $1 / (1 \text{ ps}) = 1000 \text{ GHz}$; $1 \text{ ns} = 1000 \text{ ps}$.

Why $T_c \geq t_{pcq} + t_{pd} + t_{setup}$

1. At a clock edge, flip-flop 1 launches new data. It appears at its Q after t_{pcq} .
2. The data travels through the logic, taking t_{pd} .
3. It must arrive at flip-flop 2 at least t_{setup} before the NEXT edge.
4. So the gap between edges (T_c) must cover all three: $T_c \geq t_{pcq} + t_{pd} + t_{setup}$.

Class homework: path 0.28 ns, setup 0.05 ns

1. $T_c = 0.28 + 0.05 = 0.33 \text{ ns}$
2. $f = 1 / 0.33 \text{ ns} = (1 / 0.33) \text{ GHz}$
3. $1 / 0.33 = 3.0303... \rightarrow f_{max} \approx 3.03 \text{ GHz}$

t_pcq = 40 ps, logic = 250 ps, setup = 30 ps

1. $T_c = 40 + 250 + 30 = 320 \text{ ps}$
2. $f = 1 / 320 \text{ ps} = 1000 / 320 \text{ GHz}$ (because $1/\text{ps} = 1000 \text{ GHz}$)
3. $1000 / 320 = 3.125 \rightarrow 3.125 \text{ GHz}$

Hold check: t_ccq = 30 ps, t_cd = 10 ps, t_hold = 50 ps

1. Left side: $30 + 10 = 40 \text{ ps}$
2. $40 < 50 \rightarrow \text{hold violation}$. Fix: add at least 10 ps of buffer delay on that short path.

Ring oscillator: an ODD number N of inverters in a loop → $T = 2 \cdot N \cdot t_{pd}$, $f = 1/T$.

Why $2 \cdot N$: a change travels once round the loop (N delays) to flip the output; it must go round twice to return to the starting value.

Example: 5 inverters $\times 20 \text{ ps} \rightarrow T = 2 \times 5 \times 20 = 200 \text{ ps} \rightarrow f = 1000 / 200 = 5 \text{ GHz}$.

Metastability: if D changes inside the setup/hold window, Q can hover between 0 and 1 for an unpredictable time (ball balanced on a hill), then fall either way. It cannot be eliminated for asynchronous inputs — only made rare with a **2-3 flip-flop synchronizer** (huge MTBF). Overclocking shrinks the setup margin.

6. Registers, Counters, Shift Registers

Registers

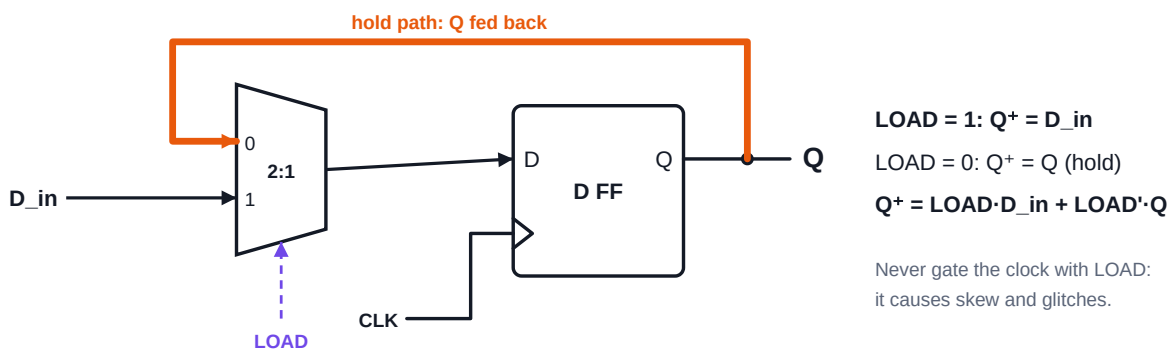
n-bit register = **n D flip-flops sharing one clock**. PC, IR, general registers and pipeline registers are all registers.

Load enable = a 2:1 MUX before each D: $Q^+ = \text{LOAD} \cdot D + \text{LOAD}' \cdot Q$ (LOAD = 0 feeds Q back → hold). Never AND the clock with LOAD (skew, glitches).

Synchronous reset: acts only at the clock edge (always @(posedge clk) if (rst)).

Asynchronous reset: acts immediately (always @(posedge clk or posedge rst)), used for power-on.

Lab priority: **reset > load > hold**.



D6.2a · Load enable: 2:1 MUX in front of each D

Counters

Counting rule: a bit toggles exactly when ALL lower bits are 1 (0111 → 1000).

Bit k runs at $f_clk / 2^{(k+1)}$: $Q_0 = f/2$, $Q_1 = f/4$, $Q_2 = f/8$.

Flip-flops for a mod-N counter = $\lceil \log_2 N \rceil$ = smallest k with $2^k \geq N$.

Ripple counter: settle time = $n \times t_{pd}$, $f_max = 1 / (n \times t_{pd})$.

Synchronous up counter with T FFs: **$T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_0 \cdot Q_1$, $T_3 = Q_0 \cdot Q_1 \cdot Q_2$** .

With D FFs: **$D_0 = Q_0'$, $D_1 = Q_1 \oplus Q_0$, $D_2 = Q_2 \oplus (Q_1 \cdot Q_0)$** . Down counter: $T_1 = Q_0'$, $T_2 = Q_0' \cdot Q_1'$ (toggle when all lower bits are 0).

How many flip-flops? (powers: 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024)

1. mod-6: $4 < 6 \leq 8 = 2^3 \rightarrow 3$ FFs
2. mod-12: $8 < 12 \leq 16 = 2^4 \rightarrow 4$ FFs
3. mod-20: $16 < 20 \leq 32 = 2^5 \rightarrow 5$ FFs
4. mod-1000 (1 MHz → 1 kHz): $512 < 1000 \leq 1024 = 2^{10} \rightarrow 10$ FFs
5. 32,768 Hz → 1 Hz: $32,768 = 2^{15} \rightarrow 15$ divide-by-2 stages

Ripple counter speed

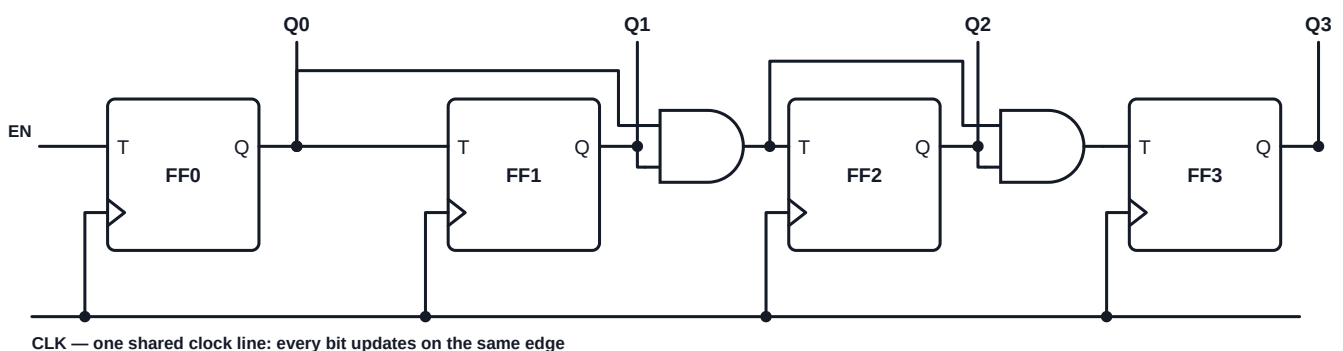
1. 4-bit, $t_{pd} = 10 \text{ ns}$: $\text{settle} = 4 \times 10 = 40 \text{ ns} \rightarrow f = 1 / 40 \text{ ns} = 1000 / 40 \text{ MHz} = 25 \text{ MHz}$
2. 16-bit, 10 ns: $160 \text{ ns} \rightarrow 1000 / 160 = 6.25 \text{ MHz}$
3. 12-bit, 0.6 ns: $12 \times 0.6 = 7.2 \text{ ns} \rightarrow 1000 / 7.2 \text{ MHz} \approx 138.9 \text{ MHz} (\approx 139 \text{ MHz})$
($1 / \text{ns} = 1000 \text{ MHz.}$)

Deriving $T0 = 1, T1 = Q0, T2 = Q0Q1$ (3-bit up counter)

1. Write present $\rightarrow$ next: 000 $\rightarrow$ 001, 001 $\rightarrow$ 010, 010 $\rightarrow$ 011, 011 $\rightarrow$ 100, 100 $\rightarrow$ 101, 101 $\rightarrow$ 110, 110 $\rightarrow$ 111, 111 $\rightarrow$ 000
2. $T = 1$ wherever the bit changes ($T = Q \oplus Q^+$)
3. $Q0$ changes on every row $\rightarrow T0 = 1$
4. $Q1$ changes only on rows where $Q0 = 1$ (001, 011, 101, 111) $\rightarrow T1 = Q0$
5. $Q2$ changes only on 011 and 111 ($Q1 = Q0 = 1$) $\rightarrow T2 = Q0 \cdot Q1$

Ripple glitch 0111 $\rightarrow$ 1000: $Q0$ falls first (0110 = 6), then $Q1$ (0100 = 4), then $Q2$ (0000 = 0), then $Q3$ rises (1000 = 8). Seen values: **7 $\rightarrow$ 6 $\rightarrow$ 4 $\rightarrow$ 0 $\rightarrow$ 8**. A synchronous counter changes all bits together — no false values.

	Ripple (async)	Synchronous
Clock	only FF0; others clocked by previous Q	one shared clock
Delay	$n \times t_{pd}$ (grows)	constant
Glitches	yes	no
Hardware	minimal	extra AND gates



$T0 = EN$ (1), $T1 = Q0$, $T2 = Q0 \cdot Q1$, $T3 = Q0 \cdot Q1 \cdot Q2$ — a bit toggles when all lower bits are 1.

D6.4a · 4-bit synchronous up-counter (T flip-flops + AND chain)

Shift registers

SISO = delay line. **SIPO** = serial $\rightarrow$ parallel (UART receiver). **PISO** = parallel $\rightarrow$ serial (UART transmitter; needs a 2:1 load/shift MUX before each D). **PIPO** = plain register/buffer. Verilog shift-left with serial input: `q <= {q[2:0], sin};`
Ring counter ($Q3 \rightarrow D0$, preset 0001): **n states**. **Johnson** ($Q3' \rightarrow D0$): **2n states**, unused = $2^n - 2n$. **LFSR** (XOR feedback, maximal): **$2^n - 1$ states** (all-zero excluded).

SIPO trace (new bit enters $q[0]$), input 1, 0, 1, 1 from 0000: 0001 $\rightarrow$ 0010 $\rightarrow$ 0101 $\rightarrow$ **1011**.

PISO trace (load 1011, shift right, $d_{out} = q[0]$): $q = 1011$ (out 1), 0101 (1), 0010 (0), 0001 (1) $\rightarrow$ the LSB leaves first.

Johnson 4-bit: 0000 $\rightarrow$ 0001 $\rightarrow$ 0011 $\rightarrow$ 0111 $\rightarrow$ 1111 $\rightarrow$ 1110 $\rightarrow$ 1100 $\rightarrow$ 1000 $\rightarrow$ 0000 (8 states; $16 - 8 = 8$ unused).

7. Register Files & FSMs

Register file

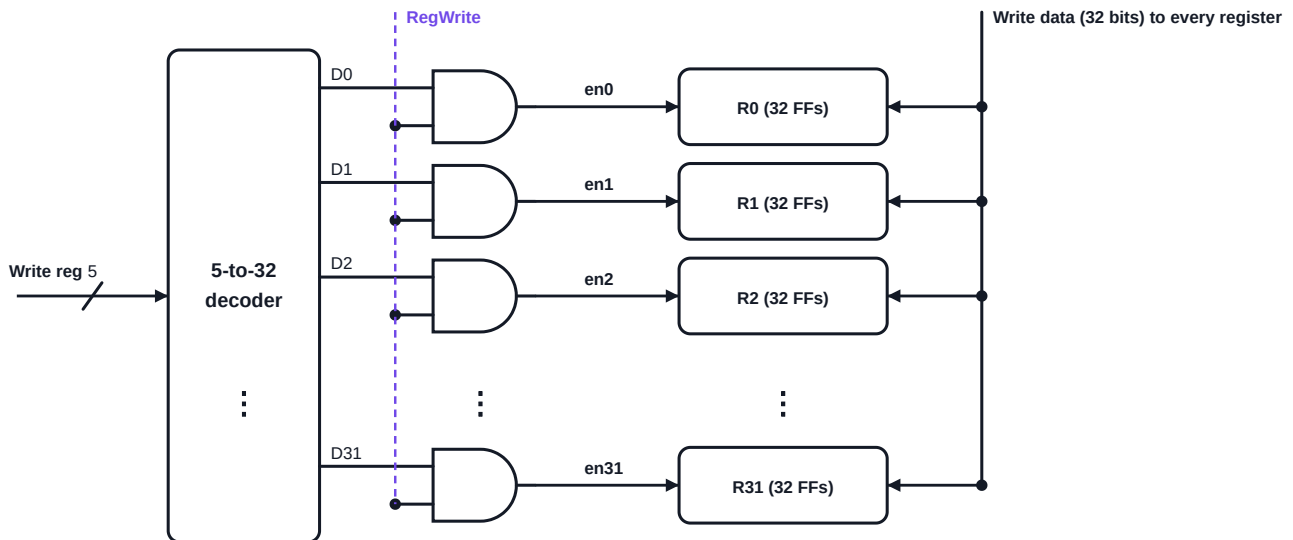
MIPS / RISC-V: **32 registers × 32 bits, 2 read ports + 1 write port** (an R-type reads 2 and writes 1 per cycle).

Address bits per port = **$\log_2(\text{number of registers})$** (32 → 5; 64 → 6; 16 → 4). Total flip-flops = registers × width (16 × 8 = 128; 32 × 32 = 1024).

Write port (synchronous): write address → n-to- 2^n decoder → each output AND RegWrite → load enable of one register; write data goes to all; only the enabled one captures it at the edge. “Decoder picks WHERE, WE picks WHETHER.”

Read ports (combinational): every register feeds TWO 32:1 MUXes; Read register 1/2 are the 5-bit selects. No clock → operands ready in the same cycle.

Register 0 (\$zero / x0) always reads 0; writes to it are ignored.



The decoder picks WHERE (one-hot), RegWrite picks WHETHER. Only the enabled register captures the data at the edge.

D7.2b · Write port: 5-to-32 decoder + RegWrite → register enables

Finite state machines

FSM = (S, I, O, **δ** , **λ**): **δ** = transition (next-state) function, **λ** = output function.

Hardware: next-state logic (combinational) → **state register** (the only flip-flops) → output logic, with the present state fed back.

Moore: output = $\lambda(\text{state})$ — written INSIDE the circle; changes only with the state; glitch-free; output one cycle later; often one more state.

Mealy: output = $\lambda(\text{state}, \text{input})$ — written ON the arrow (in/out); same-cycle output; fewer states; can glitch.

Flip-flops: binary encoding [**$\log_2 N$**]; one-hot **N**.

Design flow: understand the spec → state diagram → state table → encode states → next-state and output equations (K-maps) → build/simulate.

Sequence detector 1011 — the method

Each state remembers the longest end of the input that is still a start of 1011

1. S0 = nothing matched, S1 = "1", S2 = "10", S3 = "101" (Moore adds S4 = "1011" seen, output 1)
2. Forward edges: S0 →1→ S1 →0→ S2 →1→ S3 →1→ match
3. Fall-back edges: ask "I had X, I got bit b; what is the longest end of Xb that starts 1011?"
4. S0 on 0: "0" → nothing → S0. S1 on 1: "11" → ends in "1" → S1
5. S2 on 0: "100" → nothing → S0. S3 on 0: "1010" → ends in "10" → S2
6. S3 on 1 (match): **overlap** keeps the last "1" → S1; **non-overlap** throws it away → S0

State	Mealy overlap 0 1	Mealy no-overlap 0 1	Moore overlap 0 1 (out)	Moore no-overlap 0 1
S0	S0/0 S1/0	S0/0 S1/0	S0 S1 (0)	S0 S1
S1	S2/0 S1/0	S2/0 S1/0	S2 S1 (0)	S2 S1
S2	S0/0 S3/0	S0/0 S3/0	S0 S3 (0)	S0 S3
S3	S2/0 S1/1	S2/0 S0/1	S2 S4 (0)	S2 S4
S4	—	—	S2 S1 (1)	S0 S1 (1)

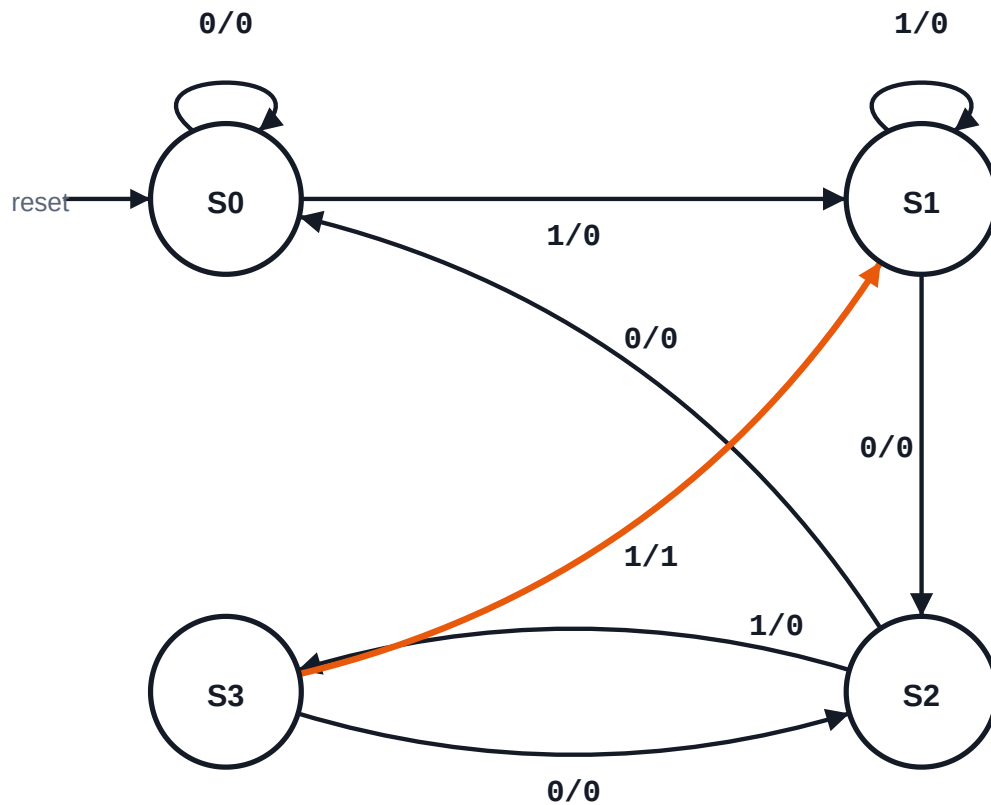
Mealy: 4 states → 2 FFs. Moore: 5 states → 3 FFs binary, 5 one-hot. Moore S4 mirrors S1 (overlap) or S0 (no overlap).

Trace Mealy overlap on 10110110

Bit #	1	2	3	4	5	6	7	8
Input	1	0	1	1	0	1	1	0
State before	S0	S1	S2	S3	S1	S2	S3	S1
Output	0	0	0	1	0	0	1	0

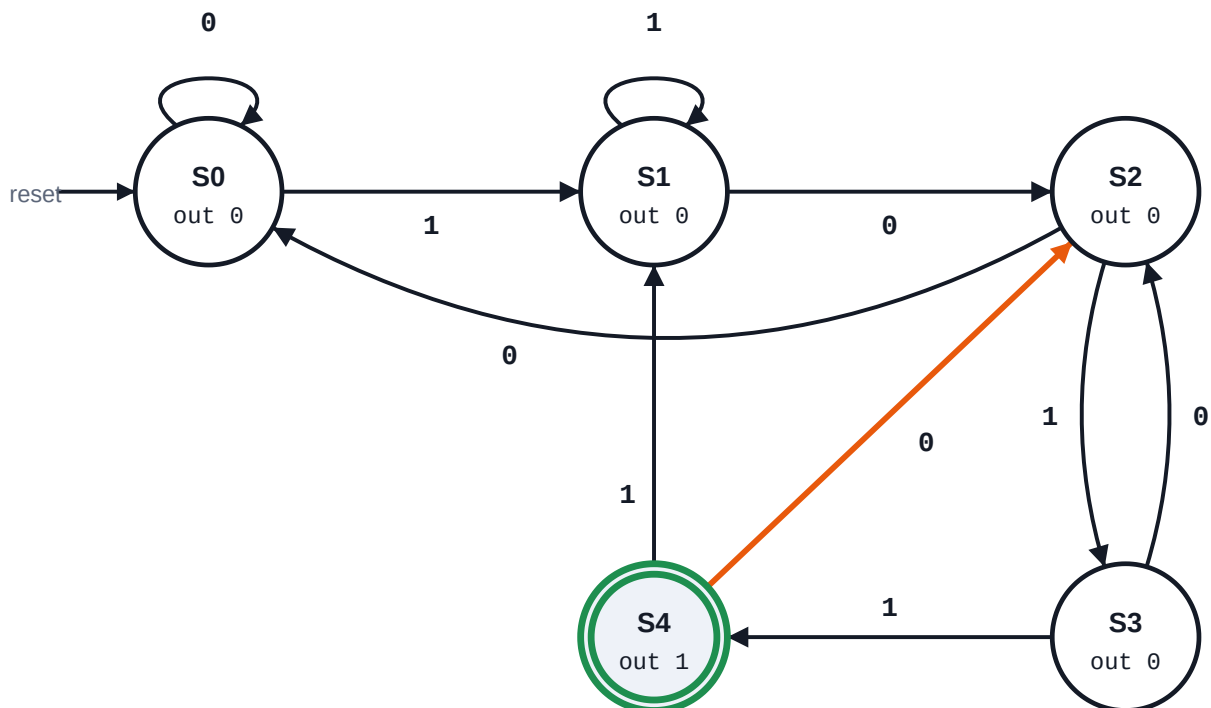
2 matches with overlap (bits 1–4 and 4–7); non-overlap gives 1 (after bit 4 it restarts at S0). Moore shows the same pulses one clock later.

Mealy 1011, overlap: S3 $\xrightarrow{1/1}$ S1 (input/output on arrows)



D7.5a · Mealy 1011 detector, overlapping

Moore 1011, overlap: S4 $\xrightarrow{0}$ S2, $\xrightarrow{1}$ S1 (output in the circle)



D7.5c · Moore 1011 detector, overlapping (5 states)

Do not reset to S0 after a match when overlap is allowed. From S3 on 0 go to S2, not S0. A Moore detector needs one more state than the Mealy one.

8. SRAM, DRAM & Memory Hierarchy

Flip-flops cost 20+ transistors per bit → too expensive for gigabytes. Core tension: **fast memory is small and expensive; big memory is slow and cheap.**

SRAM — 6T cell

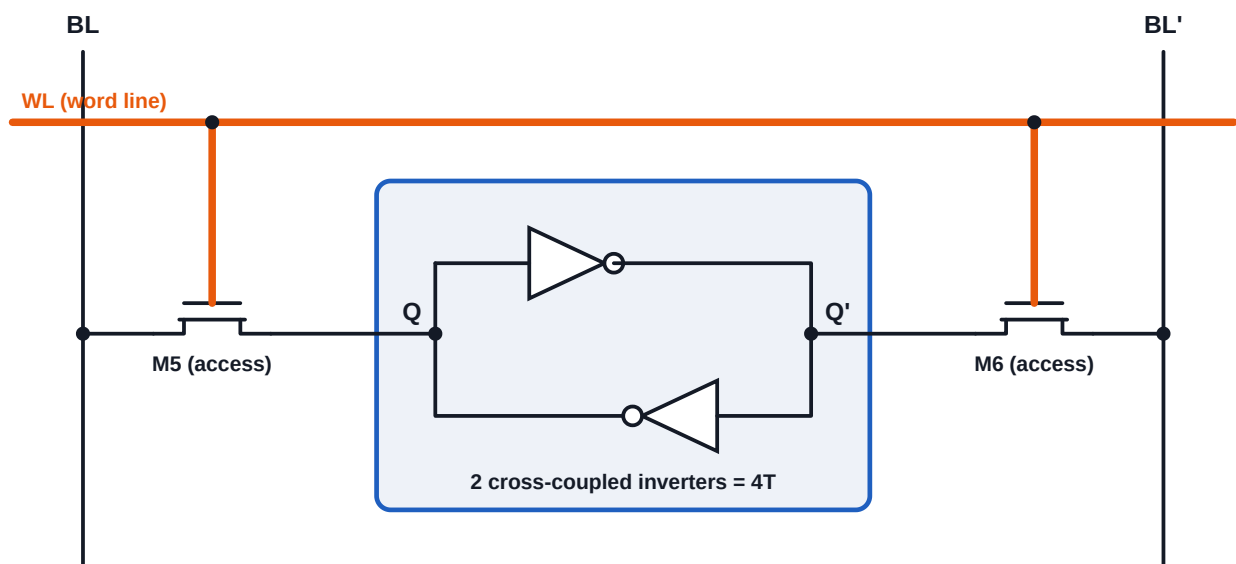
6T = 4 transistors (two cross-coupled inverters, store Q and Q') + 2 access transistors (M5, M6). Word line (WL) = select the cell; bit lines BL and BL' = data and its complement.

Static: the feedback loop holds the bit (actively) while powered — no refresh.

Read: (1) precharge both bit lines high; (2) raise WL; (3) the side storing 0 pulls its bit line slightly low; (4) sense amplifier detects which side dropped. **Non-destructive.**

Write: (1) drive BL/BL' to the new value; (2) raise WL; (3) strong drivers overpower the latch; (4) latch flips and holds.

A few ns, low density, expensive, static power → **caches, register files, buffers.**



4 transistors (2 inverters, each 1 PMOS + 1 NMOS) store the bit actively; WL = 1 connects Q to BL and Q' to BL'.

Total 6T per bit. Reads are non-destructive; the latch holds the value as long as power is on (static).

D8.2a · 6T SRAM cell

DRAM — 1T1C cell

1 access transistor + 1 storage capacitor (charged = 1, empty = 0) → about 6× denser than SRAM.

Dynamic: the charge leaks (full → lost in about 64 ms), so the controller reads and rewrites **every row about every 64 ms.**

Destructive read: (1) the capacitor shares its charge with the bit line; (2) a tiny signal; (3) sense amplifier swings it to 0/1; (4) **write-back.** Every read = read + rewrite.

50–100 ns, cheap per bit → **main memory**, GPU memory (GDDR).

Per-row refresh interval = retention time ÷ number of rows. Fraction of time busy = row refresh time ÷ interval.

64 ms, 8192 rows

1. Interval = $64 \text{ ms} \div 8192$
2. $64 \div 8192 = 0.0078125 \text{ ms}$ ($8192 = 64 \times 128$, so $64/8192 = 1/128 = 0.0078125$)
3. Convert: $0.0078125 \text{ ms} \times 1000 = \mathbf{7.8125 \mu s \approx 7.8 \mu s}$
4. If one row refresh takes 50 ns: busy fraction = $50 \text{ ns} \div 7812.5 \text{ ns} = 0.0064 = \mathbf{0.64 \%}$

	SRAM	DRAM
Cell	6 transistors	1 transistor + 1 capacitor
Storage	active (latch)	passive (charge)
Density	low	high ($\approx 6\times$)
Speed	a few ns	50–100 ns
Refresh	no	yes ($\approx 64 \text{ ms}$)
Read	non-destructive	destructive + write-back
Cost/bit	high	low
Used for	caches L1/L2/L3, register files	main memory, GDDR

Why SRAM is faster: the latch actively drives the bit lines; no destructive read/write-back or refresh; small arrays on the CPU die.

Memory hierarchy (researched)

Registers ($< 1 \text{ ns}$) $\rightarrow$ L1 ($\approx 1 \text{ ns}$) $\rightarrow$ L2 (3–10 ns) $\rightarrow$ L3 (10–40 ns) $\rightarrow$ DRAM (50–100 ns) $\rightarrow$ SSD ($\approx 100 \mu s$) $\rightarrow$ HDD ($\approx 10 \text{ ms}$). Down = bigger, slower, cheaper per bit.

Works because of **locality**: *temporal* (used again soon — loop variables) and *spatial* (neighbours used soon — arrays).

AMAT = hit time + miss rate $\times$ miss penalty.

Memory wall: CPU speed grew $\approx 50 \text{ \%/year}$, DRAM latency only $\approx 7 \text{ \%/year}$ $\rightarrow$ memory latency limits performance.

AMAT: hit 1 ns, miss rate 5 % = 0.05, penalty 80 ns $\rightarrow 1 + 0.05 \times 80 = 1 + 4 = \mathbf{5 \text{ ns}}$.

9. Von Neumann vs Harvard

A computer = **CPU + memory + I/O + bus** (bus = address + data + control lines).

Stored-program concept (von Neumann, “First Draft of a Report on the EDVAC”, 1945): the program lives in the SAME memory as data, as numbers. Change the program by loading new numbers.

Von Neumann: one memory, **one shared bus**; four classic components: control unit, memory, ALU, I/O.

Von Neumann bottleneck (term by **John Backus, 1977**): fetch and data access must take turns on one bus → contention, IPC capped, idle ALU, worse as CPUs get faster. Fixes: caches, wider buses, Harvard split.

Harvard: separate instruction and data memories + two buses → fetch AND data access in ONE cycle (up to 2× throughput). Name from the **Harvard Mark I (1944)**: instructions on punched tape, data in relays. Used in DSPs, real-time systems, microcontrollers (**ATmega328 / Arduino Uno = true Harvard**).

Modified Harvard (every modern CPU): split **L1-I / L1-D** caches (Harvard inside) over unified L2/L3 and DRAM (Von Neumann outside). ARM Cortex-M4 and x86-64 are modified Harvard.

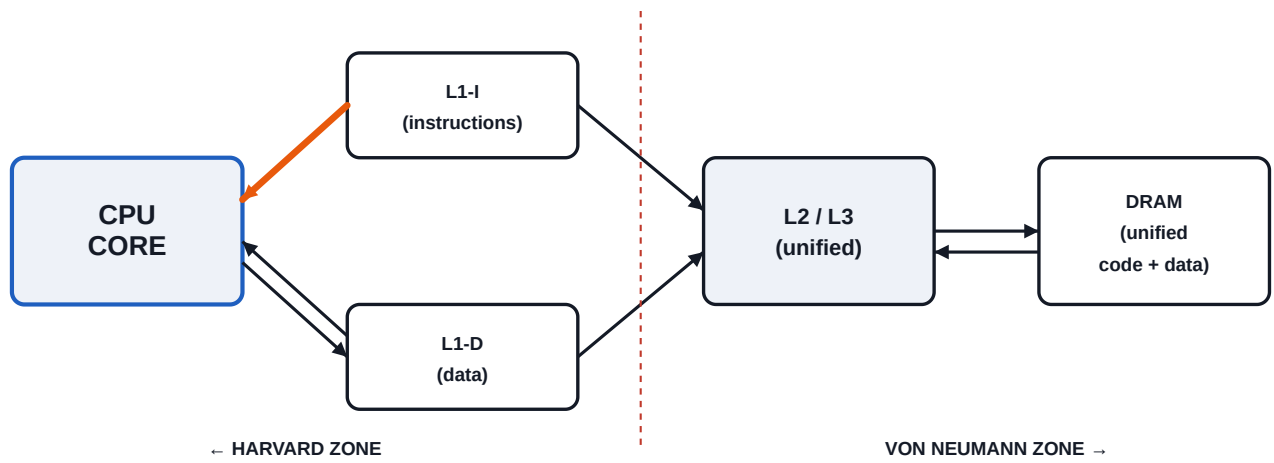
Bandwidth = bits moved per cycle × clock frequency ÷ 8 (bytes per second).

1 GHz, 32-bit instruction + 32-bit data every cycle

1. Bits per cycle = 32 + 32 = 64
2. Per second = 64 × 10⁹ bits = 64 Gbit/s
3. Bytes: 64 ÷ 8 = **8 GB/s** on the single Von Neumann bus
4. Harvard: each bus carries 32 bits × 10⁹ ÷ 8 = 4 GB/s

Homework 2 GHz, 64 + 64 bits: 128 × 2 × 10⁹ = 256 Gbit/s ÷ 8 = **32 GB/s** (Harvard 16 + 16).

	Von Neumann	Harvard
Memory	one (unified)	two (I and D)
Buses	one shared	two
Fetch + data	sequential (2 cycles)	simultaneous (1 cycle)
Throughput	bottlenecked	up to 2×
Cost	lower	higher (more pins)
Flexibility	high (code = data)	lower
Use	general CPUs	DSPs, microcontrollers



Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.

D9.5a · Split L1-I / L1-D over unified L2/L3 and DRAM

10. CISC vs RISC

ISA = the hardware/software contract (software compiles TO it, hardware EXECUTES it).

CISC: hundreds of instructions, **variable length (x86: 1-15 bytes)**, memory operands (`add [mem1], [mem2]`), complex decoder, often **microcode**, denser code. x86, VAX, 68000.

RISC: few simple instructions, **fixed 32-bit length, load/store only** (only lw/sw touch memory), simple decoder, easy to pipeline, more instructions per task. MIPS, ARM, RISC-V, SPARC.

RISC hallmarks = fixed length + load/store. Complexity moves: CISC → hardware, RISC → compiler.

Iron law: CPU time = instruction count × CPI × clock period (= $IC \times CPI \div f$).

CISC strengths	RISC strengths
denser code	simple, fast, cheap decoder
fewer instructions per task	easy, efficient pipelining
less instruction-fetch bandwidth	higher clock speeds possible
rich operations	simpler to design and verify

C = A + B; D = 2C

CISC (5): `mov ax,[A]; add ax,[B]; mov [C],ax; shl ax,1; mov [D],ax.`

RISC (6): `lw r1,A; lw r2,B; add r3,r1,r2; sw r3,C; add r4,r3,r3; sw r4,D.`

Iron law: 2 million instructions, CPI 1.5, 1 GHz → clock period = $1/10^9$ s = 1 ns → time = $2 \times 10^6 \times 1.5 \times 1$ ns = 3×10^6 ns = **3 ms**.

Why RISC pipelines well: fixed length (next instruction is always 4 bytes on); uniform format (decode in one stage); one simple step each; only lw/sw access memory (predictable hazards).

Code density is CISC's edge; ARM Thumb and RISC-V “C” (16-bit) close the gap.

History: CISC — VAX (1977), 68000 (1979), x86 (1978) → x86-64 (2003). RISC — IBM 801 → MIPS (Stanford, Hennessy, 1984), SPARC, ARM (1985), RISC-V (Berkeley, 2010). ARM won mobile (low power, licensing); x86 kept the desktop (compatibility). MIPS: PlayStation 1 (R3000A), N64, PS2. Hennessy & Patterson: 2017 Turing Award.

Modern reality: x86 decodes each CISC instruction into RISC-like **micro-ops (μops)** (e.g. `add [mem], eax` → load, add, store) and pipelines/reorders them — CISC outside, RISC inside, for compatibility plus speed. **RISC-V:** open, royalty-free, modular. **Domain-specific ISAs:** NVIDIA PTX (GPUs), Google TPU (matrix multiply).

“RISC runs fewer instructions” is false — fewer instruction TYPES, usually MORE instructions executed. The ISA alone does not decide speed.

11. MIPS Registers & Instruction Formats

Name	Number	Use
\$zero	0	always 0 (writes ignored)
\$at	1	assembler temporary
\$v0-\$v1	2-3	return values, syscall code
\$a0-\$a3	4-7	arguments
\$t0-\$t7	8-15	temporaries (caller-saved)
\$s0-\$s7	16-23	saved (CALLEE-saved)
\$t8-\$t9	24-25	temporaries
\$k0-\$k1	26-27	OS kernel
\$gp \$sp \$fp \$ra	28 29 30 31	global, stack, frame pointer, return address

\$t = “temporary, don't trust it” (caller saves if needed); \$s = “saved, it survives” (callee restores). PC, HI, LO are separate special registers.

The three formats (all 32 bits)

R: opcode(6) | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6) bits 31-26 | 25-21 | 20-16 | 15-11 | 10-6 | 5-0

I: opcode(6) | rs(5) | rt(5) | immediate(16)

J: opcode(6) | address(26)

R-type: opcode = 0, funct picks the operation; **rd = destination**, rs/rt sources. Assembly order (rd, rs, rt) ≠ binary order (rs, rt, rd).

I-type: **rt = destination for addi/lw**, data source for sw; rs = base. Immediate is **sign-extended** (andi/ori/xori zero-extend).

Effective address = rs + signext(imm).

Branch: $\text{imm} = (\text{target} - (\text{PC} + 4)) \div 4$; $\text{target} = \text{PC} + 4 + \text{imm} \times 4$.

Jump: $\text{target} = \text{PC} + 4[\text{31:28}] \parallel \text{address} \parallel 00$ (any word in the current 256 MB region).
jal also does $\$ra \leftarrow \text{PC} + 4$.

funct: add 100000 (0x20), addu 0x21, sub 100010 (0x22), and 100100 (0x24), or 100101 (0x25), xor 0x26, nor 0x27, slt 101010 (0x2A), sll 0x00, srl 0x02, sra 0x03, jr 0x08, mult 0x18, div 0x1A, mfhi 0x10, mflo 0x12.

opcode: R-type 0, j 2, jal 3, beq 4, bne 5, addi 8, addiu 9, slti 0x0A, andi 0x0C, ori 0x0D, lui 0x0F, lw 100011 (0x23), sw 101011 (0x2B), lb 0x20, lbu 0x24, sb 0x28.

Hex ↔ binary: 0 0000, 1 0001, 2 0010, 3 0011, 4 0100, 5 0101, 6 0110, 7 0111, 8 1000, 9 1001, A 1010, B 1011, C 1100, D 1101, E 1110, F 1111.

Encode add \$t1, \$t2, \$t3

1. R-type: opcode 0, funct add = 100000
2. rd = \$t1 = 9 = 01001; rs = \$t2 = 10 = 01010; rt = \$t3 = 11 = 01011; shamt = 00000
3. Place in binary order: 000000 | 01010 | 01011 | 01001 | 00000 | 100000
4. Join and regroup into 4s: 0000 0001 0100 1011 0100 1000 0010 0000
5. Each group to hex: 0 1 4 B 4 8 2 0 → **0x014B4820**

Encode lw \$t1, 16(\$sp)

1. I-type, opcode lw = 100011; rs = base \$sp = 29 = 11101; rt = destination \$t1 = 9 = 01001; imm = 16 = 0000 0000 0001 0000
2. Binary: 100011 | 11101 | 01001 | 0000000000010000
3. Regroup: 1000 1111 1010 1001 0000 0000 0001 0000 → **0x8FA90010**

Decode 0x012A4020

1. Hex to binary: 0000 0001 0010 1010 0100 0000 0010 0000
2. Split 6|5|5|5|5|6: 000000 | 01001 | 01010 | 01000 | 00000 | 100000
3. opcode 0 → R-type; funct 100000 → add; rs = 9 = \$t1; rt = 10 = \$t2; rd = 8 = \$t0
4. Write in assembly order (rd, rs, rt): **add \$t0, \$t1, \$t2**

Similarly 0x8D280004 = 100011 01001 01000 0x0004 = **lw \$t0, 4(\$t1)**; addi \$t0, \$zero, 5 = 0x20080005; li \$t0, 7 → addiu \$t0, \$zero, 7 = 0x24080007.

Branch offset: beq at 0x00400008, target 0x00400020

1. PC + 4 = 0x0040000C
 2. Difference = 0x20 – 0x0C = 32 – 12 = 20 bytes
 3. imm = 20 ÷ 4 = **5** (words). Backwards targets give negative imm (two's complement).
- Forward check: beq at 0x00400010 with imm 3 → 0x00400014 + 12 = 0x00400020.

Jump: j 0x00400018 → address field = 0x00400018 ÷ 4 = 0x100006 → word = 000010 + 26-bit 0x0100006 = **0x08100006**.

Sign-extended immediate 0xFFFFB: 1111 1111 1111 1011 → MSB 1 → negative. Invert: 0000 0000 0000 0100 = 4; add 1 → 5 → value **-5** (not 65531).

RISC-V

RV32I base ≈ 40 instructions, x0–x31 (x0 = 0), formats **R**, **I**, **S**, **B**, **U**, **J**, modular extensions **M** multiply/divide, **A** atomic, **F** single float, **D** double, **C** compressed 16-bit, **V** vector (e.g. RV32IMAFD). Open and royalty-free; MIPS is proprietary with formats R, I, J.

Assembly toolkit (Lab 11)

syscall: li \$v0, code → set \$a0 → syscall. Codes: **1 print int, 4 print string, 5 read int (→ \$v0), 8 read string, 9 sbrk, 10 exit, 11 print char**.

Pseudo-instructions: li (addiu, or lui + ori), la (lui + ori), move (addu rd, \$zero, rs), b (beq \$zero, \$zero), blt = slt \$at + bne, bge = slt \$at + beq, bgt/ble = slt with swapped operands, nop = sll \$zero, \$zero, 0.

mult → 64-bit product in HI:LO; **div** → **LO = quotient, HI = remainder** ($-17 \div 5 \rightarrow -3, -2$). **la** loads an ADDRESS, **lw** loads a VALUE.

Shifts on -16: **sll** 2 → -64; **srl** 2 (zeros in) → 0x3FFFFFFC = 1073741820; **sra** 2 (sign copied) → -4. $x10 = (x \ll 3) + (x \ll 1)$.

Alignment: lw/sw addresses must be multiples of 4. **Endianness** of 0x12345678 at B: big-endian B+0 = 12; little-endian B+0 = 78.

Directives: .data .text .word .byte .space n .asciiz (with terminating 0) .ascii .align 2 .globl main.

12. C to MIPS

Variables → registers; conditions → slt + branch; structure → labels; REVERSE the condition (branch AWAY from the body when the test fails).

Real branches: beq, bne (+ slt / slti). blt/bge/bgt/ble are pseudo.

Array element A[i] (words): address = base + 4 × i → `sll $t1, $i, 2; add $t1, $t1, $base; lw $t2, 0($t1)`.

Run body when	MIPS to skip the body
<code>a == b</code>	<code>bne a, b, Skip</code>
<code>a != b</code>	<code>beq a, b, Skip</code>
<code>a < b</code>	<code>slt t, a, b; beq t, \$zero, Skip</code>
<code>a >= b</code>	<code>slt t, a, b; bne t, \$zero, Skip</code>
<code>a > b</code>	<code>slt t, b, a; beq t, \$zero, Skip</code>
<code>a <= b</code>	<code>slt t, b, a; bne t, \$zero, Skip</code>

if / else and max of two

```
# if (a == b) c = 1; else c = 0;
    bne $t0, $t1, Else
    li  $t2, 1
    j   Done          # essential
Else: li  $t2, 0
Done:
```

```
# c = max(a, b)    ($t0 = a, $t1 = b, $t2 = c)
    slt $t3, $t1, $t0 # t3 = (b < a) = (a > b)
    beq $t3, $zero, Else
    move $t2, $t0
    j   Done
Else: move $t2, $t1
Done:
```

Without `j Done` the if-block falls into the else-block.

Loops — init before the label, test at the top, body, increment, jump back

```
# while (i < n) { sum += i; i++; }      ($t0 = i, $t1 = n, $s0 = sum)
Loop: bge $t0, $t1, End      # leave when i >= n
      add $s0, $s0, $t0
      addi $t0, $t0, 1
      j    Loop
End:
```

```
# for (i = 0; i < 10; i++) sum += i;
      li $t0, 0      # init
Loop: slti $t2, $t0, 10 # test
      beq $t2, $zero, End
      add $s0, $s0, $t0 # body
      addi $t0, $t0, 1  # increment
      j    Loop
End:
```

The class slide's while loop uses `blt $t1, $t0, End`, which runs one extra time when `i == n`. Use `bge $t0, $t1, End`.

Calling convention

Arguments **\$a0-\$a3** (more → stack); results **\$v0-\$v1**.

jal Label: `$ra ← PC + 4`, then jump. **jr \$ra**: return. Stack grows DOWN: push = `addi $sp, $sp, -8` then `sw`; pop = `lw` then `addi $sp, $sp, 8`. Keep it balanced.

Leaf (calls nothing): no need to save `$ra`; save `$s` registers only if used. **Non-leaf**: MUST save `$ra` (`jal` overwrites it) and any value needed after the call (`$a0`, `$s`).

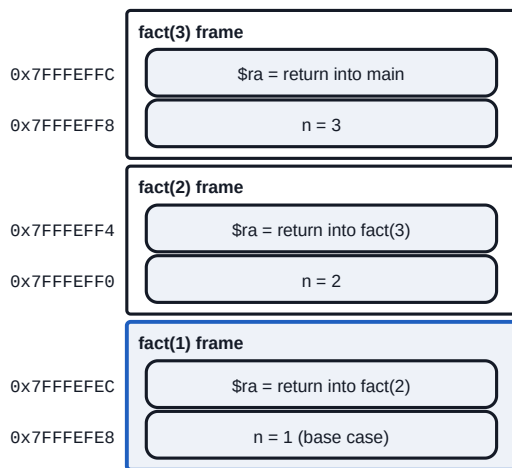
Function template: prologue (allocate, save) → base case/work → call → combine → epilogue (restore, deallocate, `jr $ra`).

Recursive factorial

```
fact: addi $sp, $sp, -8      # prologue: room for 2 words
      sw  $ra, 4($sp)      # save return address
      sw  $a0, 0($sp)      # save n
      li  $t0, 1
      ble $a0, $t0, base   # if n <= 1 return 1
      addi $a0, $a0, -1
      jal fact             # $v0 = fact(n - 1)
      lw  $a0, 0($sp)      # restore THIS level's n
      mul $v0, $a0, $v0    # n * fact(n - 1)
      j   exit
base: li  $v0, 1
exit: lw  $ra, 4($sp)      # epilogue
      addi $sp, $sp, 8
      jr  $ra
```

`fact(3)`: frames for `n = 3, 2, 1` → **3 frames × 8 bytes = 24 bytes** at peak. Unwind: `fact(1) = 1` → `fact(2) = 2 × 1 = 2` → `fact(3) = 3 × 2 = 6`.

Stack at the deepest point of fact(3)



Calls push frames downward

each: `addi $sp,$sp,-8; sw $ra,4($sp); sw $a0,0($sp)`

The unwind

1. `fact(1)` returns 1 (base case)
2. `fact(2)` restores `n = 2: 2 × 1 = 2`
3. `fact(3)` restores `n = 3: 3 × 2 = 6`

each pops 8 bytes and `jr $ra`

`$sp = 0x7FFFEFE8` (points at the last word pushed)

3 calls → 3 frames (24 bytes) at peak, then unwound one by one. Addresses assume `$sp = 0x7FFF000` before the first call.

D12.5b · `fact(3)`: frames at the deepest point and the unwind

13. Single-Cycle Datapath

Single-cycle: every instruction finishes in ONE clock cycle → **CPI = 1**, but the clock period = the SLOWEST instruction (**lw**). Each unit used once per cycle → separate instruction and data memories, extra adders.

Steps: **Fetch** ($IR \leftarrow \text{Mem}[PC]$, $PC \leftarrow PC + 4$) → **Decode** (read registers, control from opcode) → **Execute** (ALU) → **Memory** (lw/sw only) → **Write-back** (R-type, lw, addi).

Field → hardware: [31-26] opcode → Control; [25-21] rs → Read reg 1; [20-16] rt → Read reg 2 and RegDst input 0; [15-11] rd → RegDst input 1; [15-0] → sign-extend; [5-0] funct → ALU control.

MUXes: RegDst (0 = rt, 1 = rd) · ALUSrc (0 = Read data 2, 1 = sign-extended imm) · MemtoReg (0 = ALU result, 1 = memory data) · PCSrc (0 = PC + 4, 1 = branch target),

PCSrc = Branch AND Zero.

Branch target = $PC + 4 + (\text{signext}(\text{imm}) \ll 2)$.

Instr	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type	1	0	0	1	0	0	0	10
lw	0	1	1	1	1	0	0	00
sw	X	1	X	0	0	1	0	00
beq	X	0	X	0	0	0	1	01
addi	0	1	0	1	0	0	0	00

ALUOp: 00 add (lw/sw/addi), 01 subtract (beq), 10 use funct (R-type). **ALU control:** add 0010, sub 0110, AND 0000, OR 0001, slt 0111.

X rule: RegDst and MemtoReg are X whenever RegWrite = 0 (sw, beq). Never X on RegWrite, MemRead, MemWrite, Branch.

j (extension): new Jump MUX; Jump = 1; RegWrite = MemRead = MemWrite = 0; the rest X.

How to answer “Trace ... and fill the control signals”

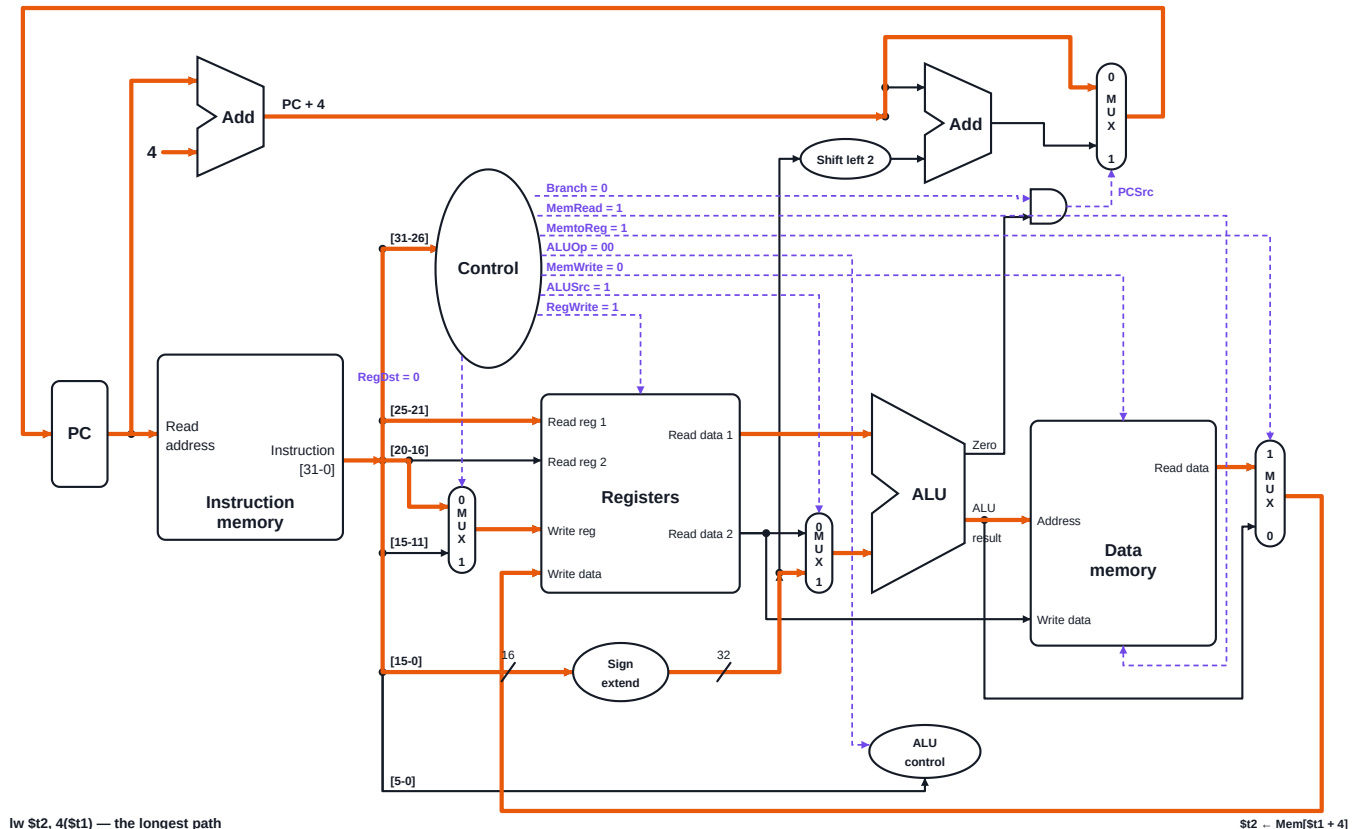
1. Highlight the fetch path: PC → instruction memory, PC → +4 adder.
2. Follow the fields: rs → Read reg 1; rt → Read reg 2 (or destination for lw/addi); rd for R-type; immediate → sign-extend.
3. At each MUX, darken the input used and write 0/1 next to it.
4. End at the destination (register file, data memory, or PC) and fill the row. **Never leave ALUOp blank** — every class sample sheet lost that mark.

lw \$t2, 4(\$t1) (0x8D2A0004): rs = \$t1 → Read data 1 → ALU; 4 → sign-extend → ALUSrc = 1 → ALU adds (ALUOp 00) → address \$t1 + 4 → MemRead = 1 → Read data → MemtoReg = 1 → Write data; destination rt → RegDst = 0; RegWrite = 1; Branch = 0, MemWrite = 0.

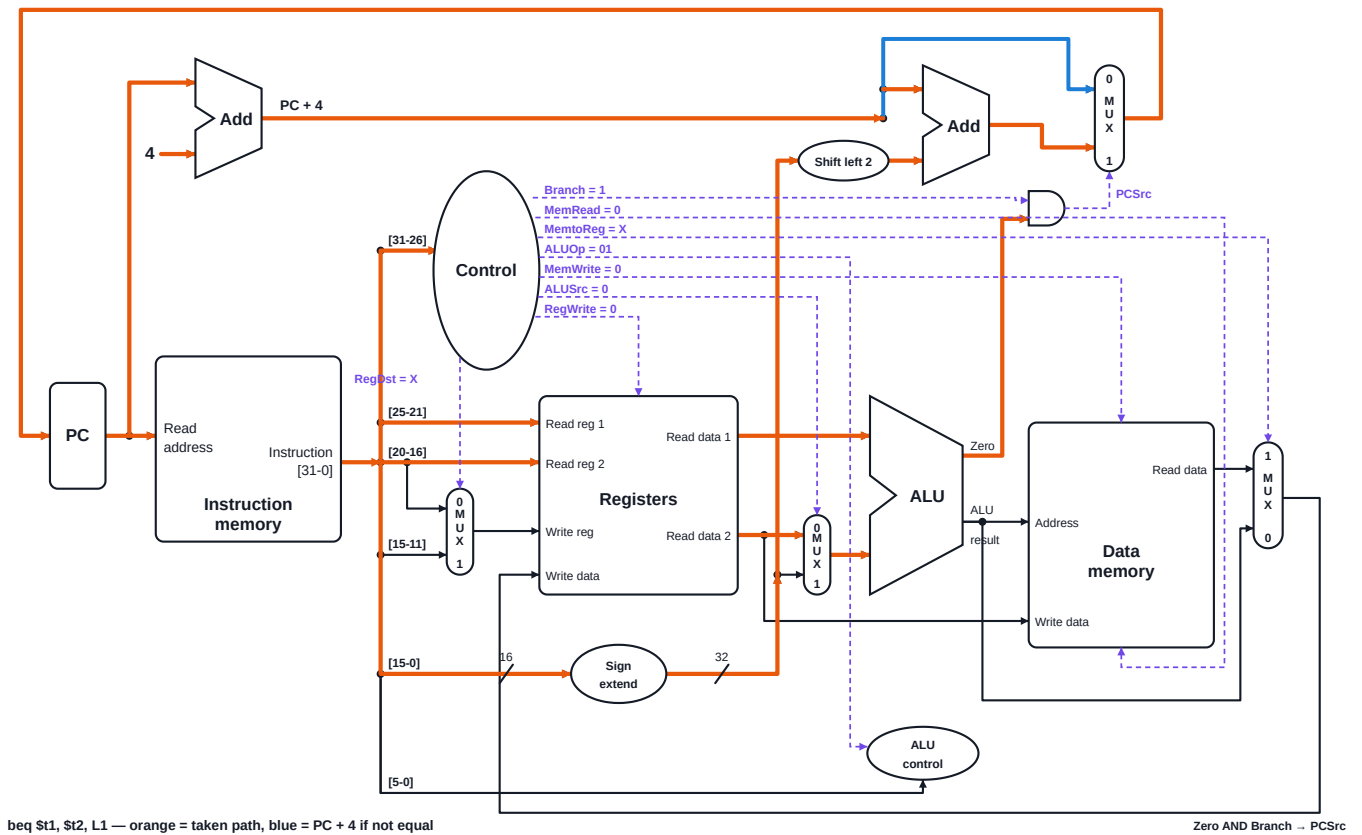
sw \$t2, 4(\$t1): same address path (ALUSrc 1, ALUOp 00); Read data 2 (\$t2) → memory
Write data; MemWrite = 1; no register written → RegWrite = 0, RegDst = X, MemtoReg = X.

beq \$t1, \$t2, L1: ALUSrc = 0, ALUOp = 01 → ALU computes $\$t1 - \$t2$; Zero = 1 if equal;
offset → sign-extend → shift-left-2 → adder with PC + 4; PCSrc = Branch (1) AND Zero. $\$t1 = 5$, $\$t2 = 5$ → Zero = 1 → jump to target. $\$t1 = 5$, $\$t2 = 7$ → Zero = 0 → PC + 4.

add \$t1, \$t2, \$t3 (R-type): ALUSrc 0, ALUOp 10 → funct 100000 → ALU control 0010;
MemtoReg 0; rd via RegDst 1; RegWrite 1; MemRead = MemWrite = 0.



D13.4b · Trace: lw \$t2, 4(\$t1)



D13.4d · Trace: beq \$t1, \$t2, L1

Critical path and clock period

Critical path = lw: instruction memory → register read → ALU → data memory → register write.

P&H delays: IMem 200, reg read 100, ALU 200, DMem 200, reg write 100 ps.

H&H: $T_c = t_{pcq} + t_{mem} + t_{RFread} + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$.

Instruction-class times (P&H)

1. lw = 200 + 100 + 200 + 200 + 100 = **800 ps** (uses every unit)
2. sw = 200 + 100 + 200 + 200 = 700 ps (no register write)
3. R-type = 200 + 100 + 200 + 100 = 600 ps (no data memory)
4. beq = 200 + 100 + 200 = 500 ps; j = 200 ps
5. Single-cycle clock = 800 ps for everyone → $f = 1000 / 800 \text{ GHz} = \mathbf{1.25 \text{ GHz}}$. beq wastes 300 ps, j wastes 600 ps.

H&H numbers: $t_{pcq} 30, t_{mem} 250, t_{RFread} 150, t_{ALU} 200, t_{mux} 25, t_{RFsetup} 20 \text{ ps}$
 → $30 + 250 + 150 + 200 + 250 + 25 + 20 = \mathbf{925 \text{ ps}}$.

14. Pipelining (bonus — researched)

Five stages: **IF** (fetch, PC + 4) · **ID** (decode, read registers) · **EX** (ALU / address) · **MEM** (lw/sw) · **WB** (write register). Pipeline registers between them: **IF/ID, ID/EX, EX/MEM, MEM/WB** (data, destination register number and control travel along). Register file: written in the first half, read in the second half of a cycle.

Cycles for n instructions in a k -stage pipeline = $k + (n - 1)$.

Speedup = $n \cdot k \div (k + n - 1) \rightarrow k$ for large n (balanced stages, no hazards).

Pipelined clock = slowest stage + pipeline-register overhead. Pipelining improves THROUGHPUT, not latency.

Why $k + n - 1$

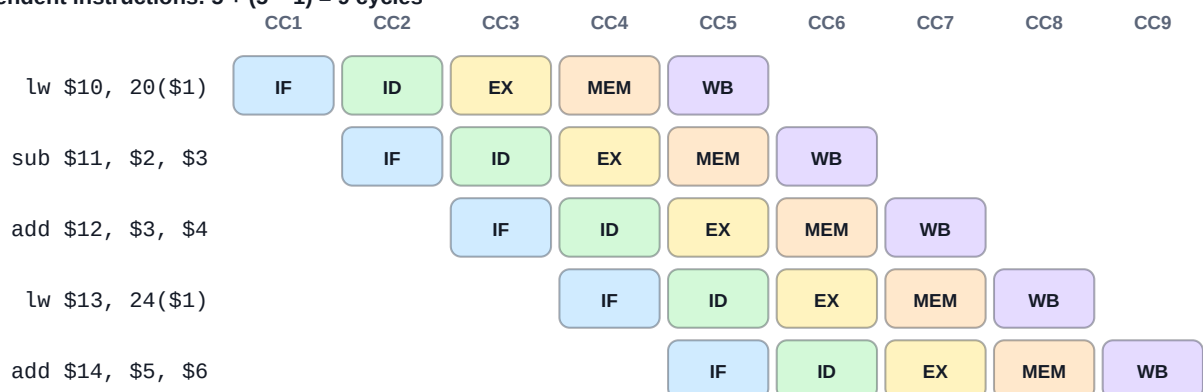
1. The first instruction needs all k cycles to come out of the last stage.
2. After that, one more instruction finishes every cycle.
3. The remaining $n - 1$ instructions add $n - 1$ cycles $\rightarrow$ total $k + n - 1$.
4. Non-pipelined: $n \cdot k$ cycles. Ratio $n \cdot k \div (k + n - 1)$; as n grows, the $k - 1$ becomes tiny next to n , so the ratio $\rightarrow k$.

P&H stages 200, 100, 200, 200, 100 ps

1. Single-cycle clock = $200 + 100 + 200 + 200 + 100 = 800$ ps
2. Pipelined clock = slowest stage = 200 ps
3. 3 lw instructions: single-cycle $3 \times 800 = 2400$ ps; pipelined $(5 + 2) \times 200 = 1400$ ps
4. Long program speedup = $800 \div 200 = 4$ (not 5, because stages are unbalanced)
5. Latency of one instruction = $5 \times 200 = 1000$ ps (longer than 800!)

With overhead: stages 250, 350, 150, 300, 200 ps + 20 ps register overhead $\rightarrow$ non-pipelined 1250 ps; clock = $350 + 20 = 370$ ps; speedup = $1250 \div 370 \approx 3.38$. Laundry (4 stages $\times$ 30 min, 4 loads): sequential $16 \times 30 = 8$ h; pipelined $(4 + 3) \times 30 = 210$ min = 3.5 h.

Five independent instructions: $5 + (5 - 1) = 9$ cycles



D14.4a · Multi-cycle pipeline diagram (no hazards)

15. Hazards (bonus — researched)

Structural: two instructions need the same hardware (one memory for IF and MEM) → fix with split I/D memory. **Data:** operand not ready yet → forwarding / stall. **Control:** next PC unknown until a branch resolves → stall, predict, delayed branch.

RAW (read after write, true dependence) is the ONLY data hazard in the in-order 5-stage pipeline, because every instruction reads in ID (stage 2) and writes in WB (stage 5) in program order. WAR and WAW appear only in out-of-order machines (fixed by register renaming).

No forwarding: back-to-back dependent instructions → **2 stalls**. **With forwarding:** ALU → ALU → **0 stalls**; load-use → **1 stall**.

Forwarding paths: **EX/MEM → EX** (producer 1 ahead) and **MEM/WB → EX** (producer 2 ahead). ForwardA/B: 00 register file, 10 EX/MEM, 01 MEM/WB.

Why 2 stalls without forwarding (add \$s0, ... then sub ..., \$s0, ...)

1. add: IF in cycle 1, ID 2, EX 3, MEM 4, **WB 5** — \$s0 written in the first half of cycle 5.
2. sub normally reads registers in ID in cycle 3.
3. It may read only in the second half of cycle 5 (after the write), so its ID must move from cycle 3 to cycle 5.
4. $5 - 3 = 2$ **stall cycles**.

Why load-use needs 1 stall even with forwarding

1. lw: EX in cycle 3, data comes out of memory at the END of MEM (cycle 4).
2. The next instruction wants it at the START of its EX, normally cycle 4.
3. Data from the end of cycle 4 cannot be used at the start of cycle 4 (that would be backwards in time).
4. Delay the user's EX to cycle 5 (**1 stall**), then forward MEM/WB → EX.

Forwarding conditions (same for Rt):

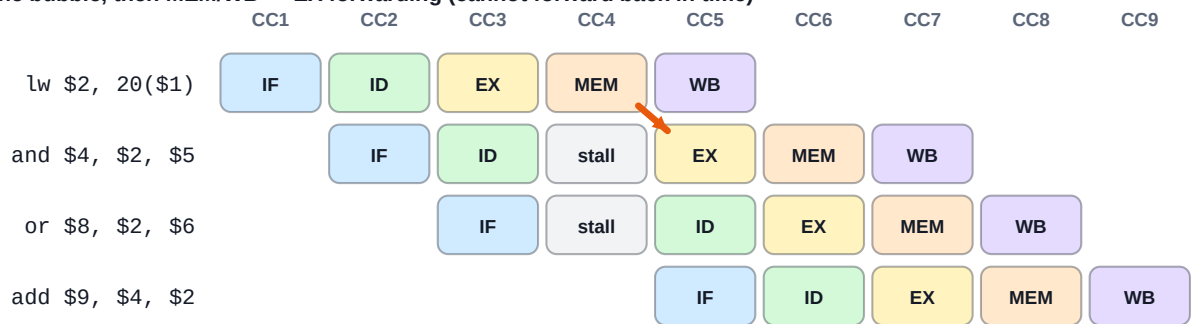
EX hazard: if EX/MEM.RegWrite and EX/MEM.Rd $\neq$ 0 and EX/MEM.Rd = ID/EX.Rs → ForwardA = 10

MEM hazard: if MEM/WB.RegWrite and MEM/WB.Rd $\neq$ 0 and NOT the EX hazard and MEM/WB.Rd = ID/EX.Rs → ForwardA = 01

Load-use detection (in ID): if ID/EX.MemRead and (ID/EX.Rt = IF/ID.Rs or ID/EX.Rt = IF/ID.Rt) → stall (hold PC and IF/ID, zero the control into ID/EX).

Stall cost: CPI = 1 + stall cycles per instruction.

Load-use: one bubble, then MEM/WB → EX forwarding (cannot forward back in time)



D15.5a · Load-use: 1 stall + forwarding

Compiler scheduling ($a = b + e$; $c = b + f$): original order lw, lw, add, sw, lw, add, sw has two load-use stalls → $5 + 6 + 2 = \mathbf{13 \text{ cycles}}$. Move the third lw up (lw, lw, lw, add, sw, add, sw) → no stalls → $5 + 6 = \mathbf{11 \text{ cycles}}$. Never move an instruction past one it depends on.

Control hazards: branch decided in MEM → 3 wrong-path instructions; decide in ID → 1-cycle penalty; predict not-taken (flush if taken); 2-bit dynamic predictors; MIPS delayed branch (the instruction after a branch always runs).

All formulas on one page

Boolean

$A + AB = A$
 $A(A + B) = A$
 $A + A'B = A + B$
 $AB + AB' = A$
 $(AB)' = A' + B'$
 $(A + B)' = A'B'$
 $XY + X'Z + YZ = XY + X'Z$
 $XOR = A'B + AB'$

K-map

Gray 00 01 11 10
group 2^k removes k variables
SOP: $1 \rightarrow X$
POS: $0 \rightarrow X$

Gate counts

NAND: NOT 1, AND 2, OR 3, XOR 4
NOR: NOT 1, OR 2, AND 3
SOP $\rightarrow$ NAND-NAND, POS $\rightarrow$ NOR-NOR

MUX

n selects $\rightarrow 2^n$ inputs
2:1 $Y = S'I_0 + SI_1$
decoder $D_k = m_k$ (2^n ANDs)

Adders

HA $S = A \oplus B$, $C = AB$
FA $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$
CLA $C(i+1) = G_i + P_i C_i$, $G = AB$, $P = A \oplus B$

Numbers

unsigned $0..2^n-1$
 2^s $-2^{n-1}..2^{n-1}-1$
 $-x = x' + 1$
MSB weight -2^{n-1}
 $A - B = A + B' + 1$
 $V = C_3 \oplus C_4$
 $CF = C_4$
sign-extend = copy MSB

Latches/FFs

SR $Q^+ = S + R'Q$ ($SR = 0$)
D latch $S = D \cdot EN$, $R = D' \cdot EN$
 $D Q^+ = D$
 $JK Q^+ = JQ' + K'Q$
 $T Q^+ = T \oplus Q$
 $T = Q \oplus Q^+$
D from T: $D = T \oplus Q$

Timing

$T_c \geq t_{pcq} + t_{pd} + t_{setup}$
 $f = 1/T_c$

hold: $t_{ccq} + t_{cd} \geq t_{hold}$
ring osc $T = 2 \cdot N \cdot t_{pd}$
 $1/ns = 1 \text{ GHz}$

Counters

FFs = $\lceil \log_2 N \rceil$
bit $k = f/2^{(k+1)}$
ripple delay $n \cdot t_{pd}$, $f = 1/(n \cdot t_{pd})$
 $T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_0 Q_1$
 $D_1 = Q_1 \oplus Q_0$, $D_2 = Q_2 \oplus Q_1 Q_0$
ring n , Johnson $2n$, LFSR $2^n - 1$

Register file

address bits = $\log_2(\text{registers})$
FFs = registers $\times$ width
load enable $Q^+ = L \cdot D + L' \cdot Q$

FSM

binary FFs $\lceil \log_2 N \rceil$, one-hot N
Mealy 1011 = 4 states, Moore = 5

Memory

refresh interval = retention $\div$ rows (64 ms / 8192 = 7.8 μ s)
AMAT = hit + miss rate $\times$ penalty

Bus

bandwidth = bits per cycle $\times f \div 8$ bytes/s

Iron law

time = IC $\times$ CPI $\times T_{clk}$

MIPS

R op|rs|rt|rd|shamt|funct (6 5 5 5 5 6)
l op|rs|rt|imm (6 5 5 16)
J op|addr (6 26)
 $EA = rs + \text{signext}(\text{imm})$
branch imm = $(\text{target} - (PC+4))/4$
jump = $PC+4[31:28] \parallel \text{addr} \parallel 00$
 $A[i] = \text{base} + 4i$
div: LO = quotient, HI = remainder

Datapath

PCSrc = Branch·Zero
ALUOp 00 add, 01 sub, 10 funct
ALU ctl add 0010, sub 0110, and 0000, or 0001, slt 0111
single-cycle clock = lw path (P&H 800 ps)

Pipeline

cycles = $k + n - 1$
speedup = $nk/(k+n-1) \rightarrow k$
clock = max stage + overhead
no-forward RAW = 2 stalls
load-use = 1 stall